

Strukture podataka i algoritmi

dr Danijel Čabarkapa
Akademija strukovnih studija Šabac
Odsek za medicinske i poslovno-tehnološke
studije

Strukture podataka i algoritmi

Uvod – o predmetu

Predavanja – nastavne teme:

1. Računarski podaci
2. Python podaci: stringovi, liste, rečnici, torke
3. Python: iskazi, petlje, funkcije, rekurzije, klase
4. Složenost i analiza algoritama
5. Nizovi i implementacije nizova
6. Liste, lančane liste, ciklične liste
7. Stekovi i redovi
8. Stabla. Binarna stabla, binarna stabla pretrage
9. Heš tabele

Predavanja – nastavne teme:

- 10. Grafovi
- 11. Algoritmi za pretraživanje
- 12. Algoritmi za pretraživanje
- 13. Algoritmi za sortiranje
- 14. Algoritmi za sortiranje
- 15. Algoritmi za rad sa tekstom

Vežbe:

1. Visual Studio Code za Python okruženje
2. Python: tipovi podataka, petlje, izrazi
3. Python: funkcije, klase, rekurzije
4. Nizovi i operacije sa nizovima
5. Liste
6. Stekovi i redovi
7. Stabla. Binarna stabla
8. Heš tabele

Vežbe:

- 9. Algoritmi pretraživanja
- 10. Algoritmi pretraživanja
- 11. Algoritmi sortiranja
- 12. Algoritmi sortiranja
- 13. Algoritmi za rad sa tekstom
- 14. Kolokvijum
- 15. Nadoknada vežbi

Sadržaj i ciljevi predmeta

Садржај предмета

Теоријска настава

Дефинисање структура података – типови података. Класификација структура података. Основне операције над структурама података. Оперативне структуре података – статичке, полустатичке и динамичке. Меморијске структуре података – операције над датотекама, методе организовања датотека, операције над записима у датотекама. Апстрактни типови података. Низови. Операције над низовима и анализа њихове ефикасности. Листа, стек и ред. Једноструко и двоструко спрегнуте листе. Имплементација стека и реда. Редови и операције над редовима. Вишедимензионални низови. Хеш табеле: појам, операције и примене хеширања. Стабла: бинарна стабла, Н-арна стабла, стабла за претраживање. Графови.

Анализа алгоритама и О-нотација. Претраживање и сортирање: линеарна и бинарна претрага Рекурзија, појам и особине рекурзије. Имплементација рекурзије. Алгоритми сортирања – Selection Sort, Bubble Sort, Insertion Sort. Алгоритми претраге – бинарна и секвенцијална претрага.

Cilj predmeta:

Upoznavanje studenata sa različitim strukturama podataka i specifičnostima vezanim za algoritme koji te strukture podataka koriste. Sticanje osnovnih znanja o algoritmima i odgovarajućim strukturama koje se koriste za pretraživanje i sortiranje, kao i njihovom praktičnom implementacijom u programskom jeziku Python.

Organizacija nastave, poeni

- ◆ Prisustvo predavanjima (evidencija na predavanjima)
- ◆ Aktivnost na računarskim vežbama (evidencija asistenta)
- ◆ SeminarSKI rad OBAVEZAN (odbrana kod asistenta)
- ◆ Jedan računarski kolokvijum (na kraju semestra)
- ◆ Pismeni ispit

#	Predispitne obaveze	Poeni
1	Prisustvo predavanjima	5
2	Aktivnost na vežbama	20
3	SeminarSKI rad	20
4	Kolokvijum	25
5	Pismeni ispit	30
	Maksimalno poena:	100

Organizacija nastave

◆ Predavanja: 3 časa nedeljno

◆ Vežbe:

- 2 časa nedeljno
- Python zadaci za razumevanje koncepata algoritama i struktura podataka
- Diskusija, analiza i konsultacije

◆ Seminarski rad:

- student samostalno piše seminarski rad na izabranu temu, asistent će ponuditi spisak tema ili je predlažu studenti
- rad se brani samostalno kod asistenta
- seminarski rad se piše prema uputstvu koje je na stranici predmeta

Organizacija nastave, poeni

◆ Kolokvijum:

- Kolokvijum nosi 25 poena
- Kolokvijum se radi u pretposlednjoj nedelji, u terminu vežbi
- Kolokvijum traje 1,5 sat
- Kolokvijum nema „prolaznost“ već koliko se poena osvoji
- Kolokvijum važi za tekuću školsku godinu
- Kolokvijum je u formi zadataka koje treba rešiti na računaru, to su modifikacije zadataka iz Python algoritama ili struktura podataka koji su rađeni na vežbama

Pismeni ispit

◆ Pismeni ispit:

- može se polagati u svakom zvaničnom ispitnom roku
- ukupno nosi 30 poena
- **3 pitanja** koja mogu biti ili teorijska ili jednostavniji zadaci (algoritmi) za proveru osnovnog znanja i razumevanja SPA
- Ispit traje 1,5 sat
- radi se bez dozvoljene literature i bilo kakvih pomagala
- za prolaz je potrebno prikupiti 50% poena sa sva tri pitanja
- pismeni ispit se polaže bez obzira na prikupljene poene predispitnih obaveza

Literatura

◆ Referentne knjige:

1. Д. Урошевић: Алгоритми и структуре података, CET, 2018.
2. Rance D. Necaise: Data structures and algorithms using Python, John Wiley & Sons, Inc. 2010, књига у електронској форми доступна на:
<https://nibmehub.com/opac-service/pdf/read/Data%20Structures%20and%20Algorithms%20Using%20Python.pdf>

◆ Univerzitetski kursevi:

- Harvard University: CS50's Introduction to Computer Science
- Michigan University: Python data structures
<https://online.umich.edu/courses/python-data-structures/>

Kontakti

- ◆ Stranica predmeta na sajtu Odseka:

<https://vmpts.edu.rs>

E-skriptarnica (SPA predmet)

- ◆ Nastavnik: dr Danijel Čabarkapa

d.cabarkapa@google.com

- ◆ Asistent: Katarina Čalić

katarinacalic005@gmail.com

Računarski podaci

1. Koncept računarskih podataka
2. Zapis brojeva u računar
3. Zapis teksta u računar
4. Obrada podataka u računar

Računarski podaci

1. Koncept računarskih podataka

Tipovi signala

- ◆ Dve osnovne kategorije signala koji se koriste u računarstvu:
 - Analogni signali
 - Digitalni signali

Signali okruženja, tipovi signala

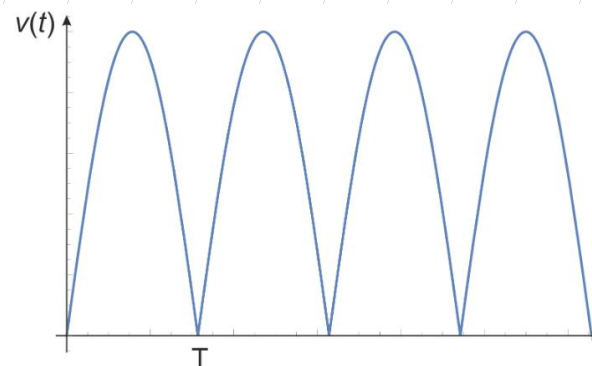
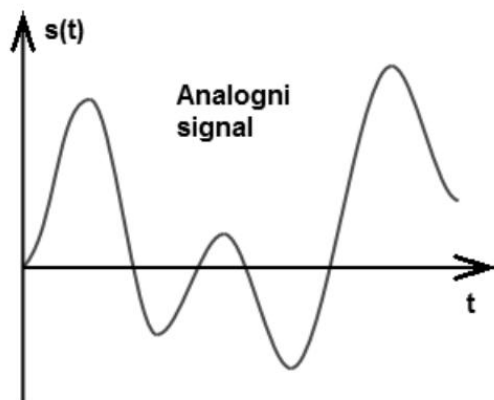
- ◆ Pod terminom signal podrazumevamo bilo koju fizičku veličinu koja se menja kao funkcija vremena, prostora ili neke druge promenljive ili promenljivih
- ◆ Signali koji nas okružuju u najvećem broju slučajeva se mogu predstaviti u obliku neprekidnih funkcija
- ◆ Da bi se pojednostavila analiza i sinteza signala, gotovo uvek je neophodno reprezentovati signale pomoću matematičkih funkcija, jedne ili više promenljivih
- ◆ Primer signala okruženja je zvučni signal koji predstavlja promenu pritiska vazduha u zadatoj tački, a kao neprekidna funkcija vremena
- ◆ Slika se može opisati intenzitetom svetlosti određene boje (tj. talasne dužine) u datom vremenskom trenutku, a kao neprekidna funkcija prostora

Signali okruženja, tipovi signala

- ◆ Iako se u praksi često sreću višedimenzionalni signali, fokus je na jednodimenzionalnim signalima, koji zavise samo od jedne nezavisne promenljive
- ◆ Iako priroda nezavisne promenljive može biti različita, uobičajeno je da ona predstavlja vreme, dok se zavisna promenljiva često naziva i amplituda signala
- ◆ Ako se posmatra skup vrednosti koje može uzeti nezavisna promenljiva (vreme), svi signali mogu se podeliti u dve grupe:
 - a) **Vremenski kontinualne signale** – kod kojih nezavisna promenljiva uzima vrednosti iz skupa realnih brojeva $\mathbb{R}$
 - b) **Vremenski diskretne signale** – kod kojih nezavisna promenljiva uzima vrednosti iz skupa celih brojeva $\mathbb{Z}$
- ◆ Kod obe grupe navedenih signala, zavisna promenljiva uvek uzima vrednosti iz skupa realnih brojeva $\mathbb{R}$
- ◆ Kontinualni signali često se nazivaju i analogni signali

Analogni signali

- ◆ Analogni zapis predstavlja analogiju između signala koji je zapisan i svojstva medijuma na kojem je signal zapisan
- ◆ Kontinualne promene signala koji se zapisuje opisuju se kontinualnim promenama materijala od kojeg je medijum sastavljen (magnetna traka, papir, hard disk, optički disk...)
- ◆ **Analogni signal** je signal koji u proizvoljnom trenutku može imati bilo koju vrednost u okviru datog opsega. Pošto se u elektronskim kolima obično koriste jednosmerni izvori napajanja, naponski opsezi signala su po pravilu ograničeni naponom napajanja
- ◆ Periodični analogni signali se ponavljaju u istim sekvencama (periodama T) tokom vremena i mogu se predstaviti matematičkim funkcijama



Periodični analogni signal

Analogni signali

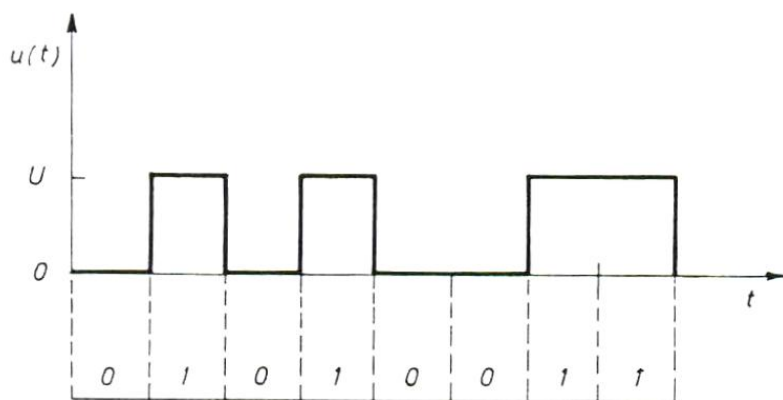
- ◆ Prednosti analognih signala: Tehnologija njihovog generisanja i prenosa je relativno jednostavna, ukoliko se zadovoljimo niskim kvalitetom
- ◆ Kvalitet zapisa signala zavisi od kvaliteta medijuma i teško je napraviti kvalitetan zapis, kao i što je teško napraviti dva identična zapisa istog signala (gubljenje kvaliteta tokom kreiranja kopija, težak prenos signala na daljinu)
- ◆ Inherentna nestalnost medijuma, njegova promenljivost tokom vremena i podložnost spoljašnjim uticajima vodi do neizbežnog pada kvaliteta analogno zapisanog signala
- ◆ Obrada analogno zapisanih signala je najčešće veoma komplikovana i za svaku vrstu obrade signala potrebno je da postoji uređaj koji je specijalizovan za tu vrstu obrade
- ◆ Analogni signali: zvuk (govor, muzika...), temperatura, pritisak vazduha, kao i velika većina drugih prirodnih signala

Digitalni signali

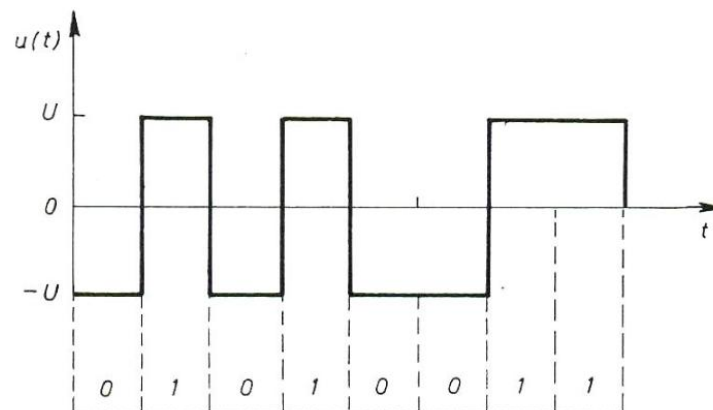
- ◆ **Digitalni signal** u svakom vremenskom trenutku može imati jednu od nekoliko unapred određenih diskretnih vrednosti
- ◆ **Binarni digitalni signal** tokom vremena može imati svega dve vrednosti: visoku vrednost (logička jedinica) i nisku vrednost (logička nula)
- ◆ Signal koji je definisan samo u nekim diskretnim trenucima vremena naziva se **vremenski diskretni signal**
- ◆ Digitalni signali se mogu predstaviti kao niz diskretnih vrednosti, u određenom trenutku mogu imati jednu vrednost iz konačnog skupa vrednosti
- ◆ Digitalni signali predstavljaju informacije u nivoima. Nivoi su diskretni opsezi analognih vrednosti. Sve vrednosti signala unutar nivoa predstavljaju isto informaciono stanje
- ◆ Informacija predstavljena jednim nivoom signala se naziva **simbol**
- ◆ Binarni signal u bilo kom trenutku predstavlja jednu binarnu cifru – bit (***Binary digit***). Kod binarnog signala, simbol predstavlja jedan bit

Digitalni signali

- ◆ Digitalni zapis predstavlja diskretnu aproksimaciju polaznog signala
- ◆ Vrednost signala se izmeri u određenim vremenskim trenucima ili tačkama prostora i na medijumu se zapišu izmerene vrednosti
- ◆ Na taj način je digitalni signal predstavljen nizom brojeva koji se nazivaju odbirci ili semplovi (*sample*)



Unipolarni digitalni signal

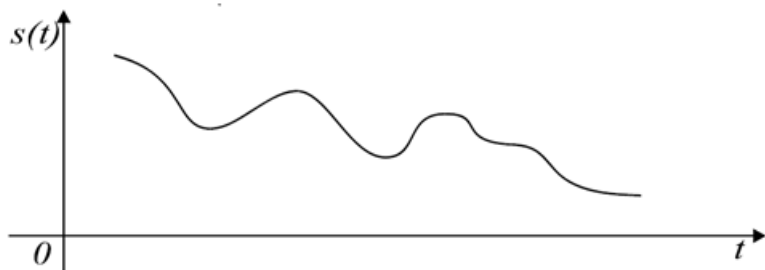


Bipolarni digitalni signal

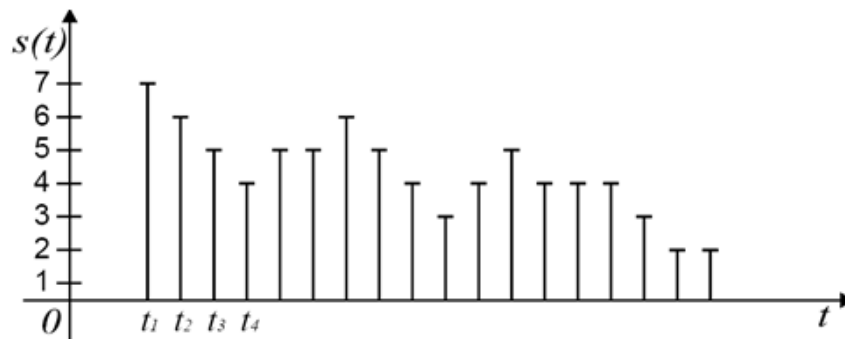
Digitalni signali

- ◆ Analogni signal se može digitalizovati pomoću **analogno-digitalnog konvertora (A/D)**, pri čemu A/D konvertor očitava vrednost signala u konstantnim periodima
- ◆ Dobijeni digitalni signal ima diskretizovanu (kvantovanu) amplitudu i predstavljen je u diskretnim vremenskim intervalima
- ◆ Frekvencija kojom A/D konvertor očitava analogni signal se naziva frekvencija semplovanja. Frekvencija semplovanja mora biti najmanje dvostruko veća od frekvencijskog opsega signala (Nikvist-Šenonova teorema odmeravanja)
- ◆ Zbog kvantizacije signala, relativno male promene analognog signala ne utiču na nivo digitalnog signala
- ◆ Digitalni signali koji imaju veći broj nivoa prenose veću količinu informacija, ali su podložniji uticaju šumova
- ◆ Binarni signal prenosi najmanju količinu informacija (jedan bit) i šumovi najmanje utiču na njih

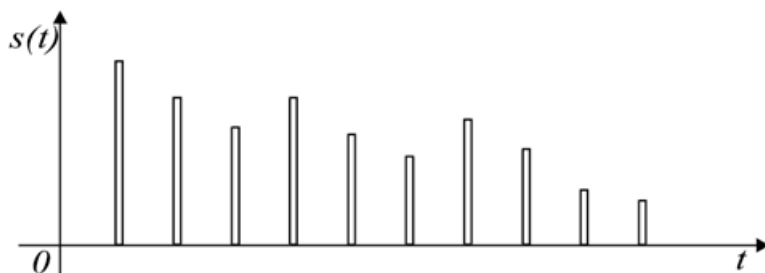
Analogno-digitalna konverzija (A/D)



Analogni (kontinualni) signal



Kvantovanje



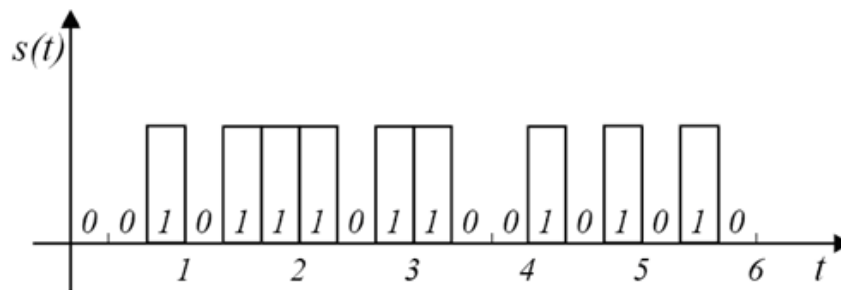
Odmeravanje

t	$s(t)$
1	001
2	011
3	101
4	100
5	101
6	010

Kodovanje

Postupak analogno-digitalne konverzije signala (diskretizacije signala) podrazumeva 3 faze:

- 1) Diskretizacija po vremenu (odmeravanje)
- 2) Diskretizacije signala po trenutnim vrednostima (kvantovanje)
- 3) Kodovanje ili predstavljanje diskretnih vrednosti signala grupom cifara (0 i 1), odnosno impulsa



Digitalni signal

Digitalni signali

- ◆ Postoji veliki broj prednosti digitalnog signala i znatno manje nedostataka
- ◆ Kvalitet reprodukcije digitalnog zapisa ne zavisi od toga kakav je kvalitet medijuma na kojem se podaci zapisuju, sve dok je medijum dovoljno dobrog kvaliteta da se zapisani brojevi mogu razaznati
- ◆ Kvarljivost koja je inherentna za sve medijume postaje nebitna jer ne utiče na kvalitet zapisa digitalnog signala, sve dok se brojevi 1 i 0 mogu pročitati kvalitet zapisa je isti
- ◆ Digitalnim zapisom moguće je kreiranje apsolutno identičnih kopija, što dalje omogućava prenos podataka na daljinu sa minimalnim gubicima
- ◆ Obrada digitalno zapisanih podataka se svodi na matematičku manipulaciju brojevima i ne zahteva korišćenje specijalizovanih uređaja
- ◆ Osnovni problem digitalnog zapisa – napredna tehnologija da bi se formirao upotrebljiv zapis (npr. elektronski sklop kojom se vrši odabiranje audio signala 40.000 puta/sekundi je prilično složen)
- ◆ Kada se došlo do tehnološkog nivoa koji je omogućavao digitalni zapis, on je doneo neverovatno brz dalji razvoj tehnologije komunikacija i računarstva

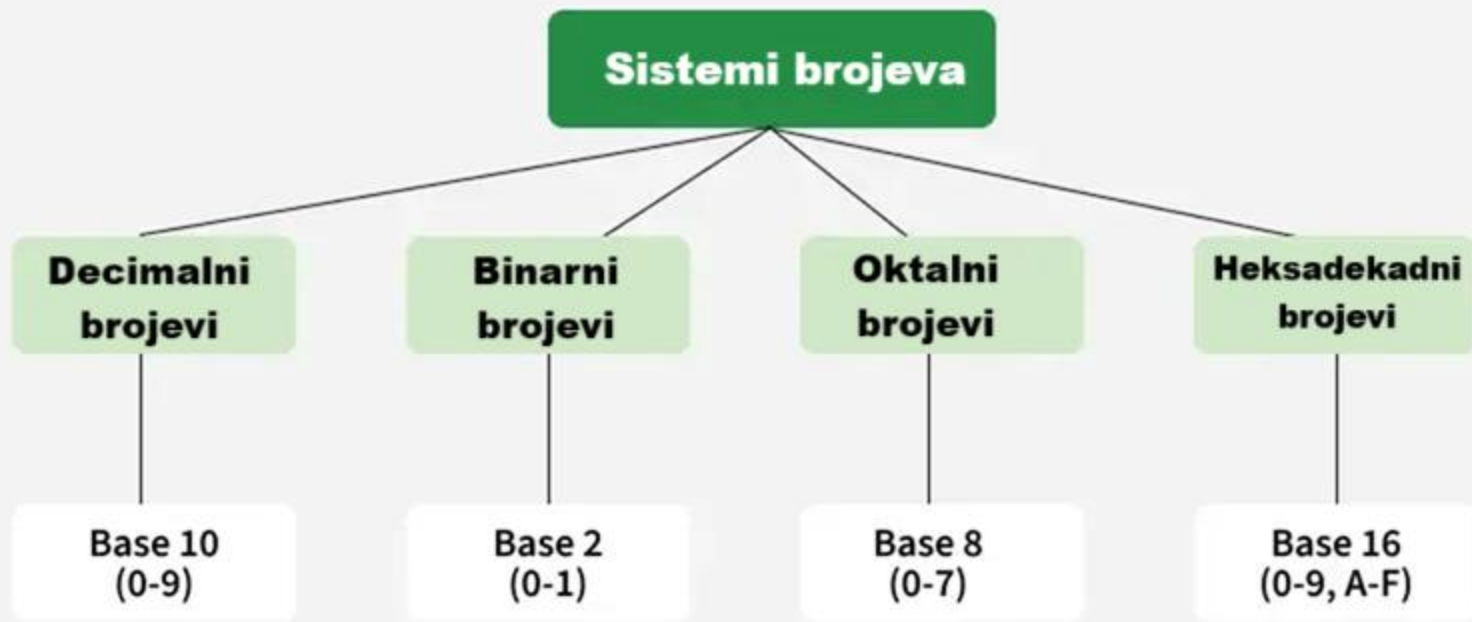
Računarski podaci

2. Zapisi brojeva u računar

Zapis brojeva u računaru

- ◆ Proces digitalizacije predstavlja postupak predstavljanja (raznovrsnih) podataka u vidu BROJEVA
- ◆ Potrebno je precizno definisati zapisivanje različitih vrsta brojeva, jer računar brojeve „vidi“ isključivo kao digitalne „1“ i „0“
- ◆ Dekadni sistem brojeva najviše se koristi u svakodnevnoj primeni, ali ga računari „ne razumeju“ dok brojeve ne prevedu u jedinice (1) i nule (0)
- ◆ U skladu sa tehnološkom osnovom digitalnih računara, osnovni sistem predstavljanja brojeva je **BINARNI** sistem (sa osnovom ili bazom 2)
- ◆ Pored binarnog, u upotrebi su **OKTALNI** i **HEKSADEKADNI** brojevni sistemi

Zapisi brojeva – brojevni sistemi



Dekadni sistem brojeva

- ◆ Dekadni sistem brojeva ima osnovu (*base*) $n=10$, pa je skup cifara ovog sistema: $S = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
- ◆ Pozicija cifre se računa od desne ka levoj strani zapisa, sa početnom pozicijom 0 i koristi se kao eksponent osnove 10

pozicije cifara

3	2	1	0	-1	-2
5	1	8	4	,3	5

- ◆ Primer: Celi broj 2345 će se računati kao:

$$2345 = 2 \times 10^3 + 3 \times 10^2 + 4 \times 10^1 + 5 \times 10^0$$

- ◆ Za slučaj decimalnog broja 1845,34 zapis je:

$$1845,34 = 1 \times 10^3 + 8 \times 10^2 + 4 \times 10^1 + 5 \times 10^0 + 3 \times 10^{-1} + 4 \times 10^{-2}$$

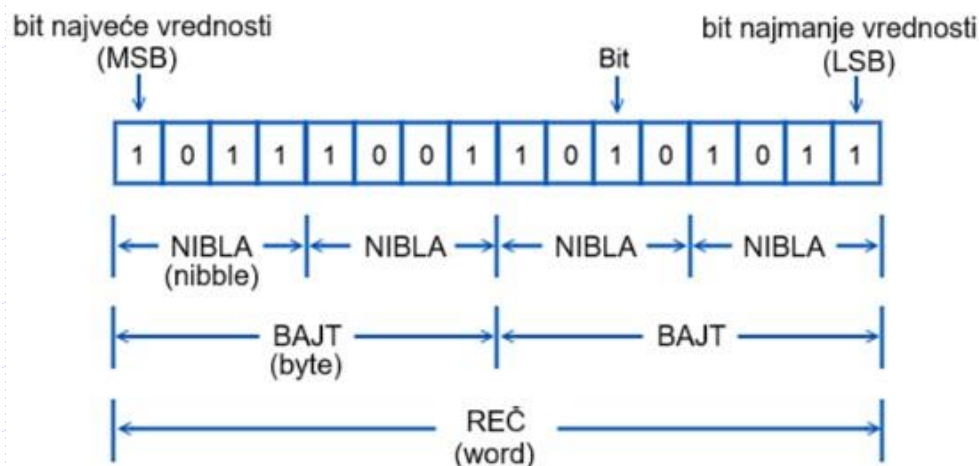
- ◆ Pozicije cifara iza decimalnog zareza se računaju redom kao -1, -2 itd...

Binarni sistem brojeva

- Binarni sistem brojeva ima osnovu $n=2$ pa je skup cifara $S = \{0, 1\}$

Aritmetika rada sa podacima u računarima zasnovana je na binarnom sistemu brojeva

- Binarni sistem se u računarima jednostavno tehnički realizuje, pošto postoje samo dve cifre 0 i 1, tako da se za postizanje prekidačkih stanja 0 ili 1 koriste brza digitalna elektronska prekidačka kola



Forme binarnih podataka:

1 bit

4 bita – nibla

8 bitova – bajt (B)

16 bitova – binarna reč

1KB=1024B

1MB=1024KB

1GB=1024MB

1TB=1024GB

....

Oktalni sistem brojeva

- ◆ Oktalni system brojeva ima osnovu $n=8$, pa je skup cifara sistema: $S = \{0,1,2,3,4,5,6,7\}$
- ◆ U prethodnim generacijama računara oktalni sistem je imao značajnu ulogu
- ◆ Primer: Broj 677 u oktalnom sistemu preveden u dekadni je:
$$(677)_8 = 6 \times 8^2 + 7 \times 8^1 + 7 \times 8^0 = 384 + 56 + 7 = (447)_{10}$$

Oktalni zapis	Binarni zapis
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

Pošto postoji 8 cifara oktalnog zapisa (od 0 do 7), dovoljne su 3 binarne cifre za zapis jednog oktalnog broja jer je $2^3=8$

Heksadekadni sistem brojeva

Decimal	Binary	Hexadecimal
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

- Heksadekadni brojni sistem ima osnovu $n=16$, pa je skup cifara ovog sistema: $S = \{0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F\}$
- Najveći broj je 15 ili slovo F (koje se predstavlja binarnom tetradom 1111)

$$(B6C98)_{16} = B \times 16^4 + 6 \times 16^3 + C \times 16^2 + 9 \times 16^1 + 8 \times 16^0 = \\ 11 \times 65536 + 6 \times 4096 + 12 \times 256 + 9 \times 16 + 8 \times 1 = \\ 72896 + 24576 + 3072 + 144 + 8 = (748696)_{10}$$

$$(1011 \ 0110 \ 0111 \ 1100 \ 0010 \ 1001 \ 1111 \\ 0001)_2 = (B67C29F1)_{16}$$

Pošto postoji 16 cifara heksadekadnog zapisa (od 0 do F), dovoljne su 4 binarne cifre za zapis jednog hekso-broja jer je $2^4=16$, binarni zapis izdeliti na grupe po 4 cifre

Konverzije brojeva iz jednog sistema u drugi

- Konverzija binarni > dekadni sistem broejva:

$$(111001111)_2 = 1 \times 2^8 + 1 \times 2^7 + 1 \times 2^6 + 0 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 256 + 128 + 64 + 8 + 4 + 2 + 1 = (463)_{10}$$

$$(101100.11)_2 = 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} = (44,75)_{10}$$

- Konverzija dekadni > binarni sistem broejva:

$$(143)_{10} = (?)_2$$

143	:2
71	1
35	1
17	1
8	1
4	0
2	0
1	0
0	1

Rezultati
deljenja

Kriterijum
kraja

Ostaci pri
deljenju

$$(143)_{10} = (10001111)_2$$

Smer
očitavanja

$$(126)_{10} = (1111110)_2$$

$$(35)_{10} = (100011)_2$$

Konverzije brojeva iz jednog sistema u drugi

- Konverzija dekadni > binarni, oktalni, heksadekadni sistem brojeva:

$$(126)_{10} = (?)_2$$

126	:2
63	0
31	1
15	1
7	1
3	1
1	1
0	1

$$(126)_{10} = (1111110)_2$$

$$(181)_{10} = (?)_8$$

181	:8
22	5
2	6
0	2

$$(181)_{10} = (265)_8$$

$$(181)_{10} = (?)_{16}$$

181	:16
11	5
0	11(B)

$$(181)_{10} = (B5)_{16}$$

heksadekadna cifra B

Konverzije brojeva iz jednog sistema u drugi

- Konverzija oktalni > binarni sistem brojeva:

Konverzija oktalnog u binarni broj vrši se jednostavnom zamenom odgovarajuća tri bita iz tabele za svaku oktalnu cifru u broju

1	4	7
001	100	111

$$(147)_8 = (001100111)_2$$

$$(54,12)_8 = 101|100,001|010 \\ = (101100,001010)_2$$

- Konverzija binarni > oktalni sistem brojeva:

Kod konverzije binarnog u oktalni broj, binarne cifre se grupišu u grupe od po 3 cifre, počev od bitova najmanje težine. Ako ukupan broj bitova nije deljiv sa tri, onda se dopisuje potreban broj vodećih nula.

$$(11111010001010)_2 = (011\ 111\ 010\ 001\ 010)_2 \\ = (37212)_8$$

$$(11010100100)_2 = (011\ 010\ 100\ 100)_2 \\ = (3244)_8$$

$$\begin{array}{c|c|c} 2^2 & 2^1 & 2^0 \\ 0 & 1 & 0 \end{array} \bigg| \begin{array}{c|c|c} 2^2 & 2^1 & 2^0 \\ 1 & 1 & 1 \end{array} \bigg| \begin{array}{c|c|c} 2^2 & 2^1 & 2^0 \\ 1 & 1 & 0 \end{array} (2) = 276_{(8)}$$

Konverzije brojeva iz jednog sistema u drugi

- Konverzija heksadekadni > binarni sistem brojeva:

Konverzija heksadekadnog u binarni broj vrši se jednostavnom zamenom odgovarajuća četiri bita iz tabele za svaku heksadecimalnu cifru u broju

0	A	B	C	D
0000	1010	1011	1100	1101

$$(A3)_{16} = (1010\ 0011)_2$$

$$3A8_{(16)} = 0011|1010|1000_{(2)}$$

- Konverzija binarni > heksadekadni sistem brojeva:

Kod konverzije binarnog u heksadekadni broj, binarne cifre se grupišu u grupe od po 4 cifre, počev od bitova najmanje težine. Ako ukupan broj bitova nije deljiv sa četiri, onda se dopisuje potreban broj vodećih nula.

$$\begin{aligned}(1001111000111000)_2 &= (?)_{16} \\ &= (1001\ 1110\ 0011\ 1000)_2 \\ &= (9E38)_{16}\end{aligned}$$

Konverzije brojeva iz jednog sistema u drugi

- Konverzija heksadekadni > oktalni (i obrnuto) sistem brojeva:

Ako je neophodno vršiti konverziju broja iz heksadekadnog u oktalni brojevi prikaz, ili obratno, lakše je koristiti binarnu decimalnu formu kao međukorak

$$\begin{aligned}(1A8E)_{16} &= (?)_8 \\ &= (0001\ 1010\ 1000\ 1110)_2 \\ &= (001\ 101\ 010\ 001\ 110)_2 \\ &= (1\ 5\ 2\ 1\ 6)_8\end{aligned}$$

$$316_{(8)} = 0|1100|1110_{(2)} = CE_{(16)}$$

$$58A_{(16)} = 0101/1000/1010_{(2)} = 10110001010_{(2)}$$

$$010/110/001/010$$

$$010 \text{ je } 2^2 \cdot 0 + 2^1 \cdot 1 + 2^0 \cdot 0 = 2$$

$$110 \text{ je } 2^2 \cdot 1 + 2^1 \cdot 1 + 2^0 \cdot 0 = 6$$

$$001 \text{ je } 2^2 \cdot 0 + 2^1 \cdot 0 + 2^0 \cdot 1 = 1$$

$$010 \text{ je } 2^2 \cdot 0 + 2^1 \cdot 1 + 2^0 \cdot 0 = 2$$

$$\longrightarrow 58A_{(16)} = 10110001010_{(2)} = 2612_{(8)}$$

Konverzija decimalnog broja u binarni

- ◆ Kod pretvaranja decimalnog broja u binarni (osnova $n=2$), fokus je na razlomljenom delu broja
- ◆ Vrš se uzastopno množenje razlomljenog dela broja sa brojem 2
- ◆ Dobijeni celobrojni deo se u svakom koraku pretvaranja, pretvara u cifru 1 ili 0 binarnog sistema
- ◆ U svakom narednom koraku, sa 2 se množi samo razlomljeni deo broja, koji je dobijen u prethodnom koraku
- ◆ Postupak se završava kada je rezultat množenja jednak nuli, ili kada se dobije broj cifara m , koji garantuje da za grešku G pri pretvaranju važi da je: $G < 2^{-m}$
- ◆ Broj u binarnom sistemu se dobija kao niz cifara koje predstavljaju celobrojne delove brojeva dobijenih uzastopnim množenjem, pri čemu je redosled očitavanja dobijenih celobrojnih cifara odozgo na dole

Konverzija decimalnog broja u binarni

$$(0,203125)_{10} = (?)_2$$

Celobrojni delovi
rezultata množenja

Smer
očitavanja

0, 203125	*2
0, 40625	
0, 8125	
1, 625	
1, 25	
0, 5	
1, 0	

$$(0,203125)_{10} = (0,001101)_2$$

Ako stanemo nakon četvrtog koraka, tada usvajamo da je 0,0011 aproksimativno jednak 0,20315, dok je zapravo taj broj jednak 0,1875

0, 203125
0, 40625
0, 8125
1, 625
1, 25
0, 5
1, 0

$$\begin{aligned} 0,0011 &= 0 \cdot 2^0 + 0 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} + 1 \cdot 2^{-4} \\ &= 0 \cdot 2^0 + 0 \cdot 2^{-1} + 0 \cdot 2^{-2} + \frac{1}{8} + \frac{1}{16} \\ &= 0,1875 \end{aligned}$$

$$G = 0,20315 - 0,1875 = 0,01565 < 2^{-4} = 0,0625$$

Broj $(37,625)_{10}$ konverovati u binarni oblik:

Celobrojni deo broja

37	:2
18	1
9	0
4	1
2	0
1	0
0	1

Razlomljeni deo broja

0, 625
1, 25
0, 5
1, 0

*2

$$(37,625)_{10} = (100101,101)_2$$

Zapisi brojeva u računaru

- ◆ U pozicionom sistemu brojeva, udeo cifre u celokupnoj vrednosti zapisanog broja zavisi od njene pozicije
- ◆ Zapis broja u računaru je samo konvencija jer brojevi koji se zapisuju su apstraktni i ne zavise od konkretnog zapisa
- ◆ S obzirom na to da je svaki zapis broja u računaru ograničen, ne mogu se zapisati svi celi ili svi realni brojevi i to se uvek prilikom programiranja mora uzimati u obzir
- ◆ Sa n bitova možemo da predstavimo označene brojeve (sa znakom broja) u opsegu: $-2^{n-1} \div 2^{n-1}-1$
- ◆ Arhitektura računara, odnosno dužina registara za podatke određuje opseg prikaza brojeva u računarima
- ◆ Opsezi zapisa celih brojeva su sledeći:
 - kod 32-bitnih računara: od -2^{31} do $2^{31}-1$
 - kod 64-bitnih računara: od -2^{63} do $2^{63}-1$

Zapisi neoznačenih brojeva

- ◆ Pod neoznačenim (*nonsigned*) brojevima podrazumeva se zapis nenegativnih celih brojeva sa izostavljenim znakom +
- ◆ Vrednost neoznačenog broja zapisanog u osnovi b definiše se na sledeći način:

$$(a_n a_{n-1} \dots a_1 a_0)_b = \sum_{i=0}^n a_i \cdot b^i = a_n \cdot b^n + a_{n-1} \cdot b^{n-1} + \dots + a_1 \cdot b + a_0$$

- ◆ Primeri:

$$(9876)_{10} = 9 \cdot 10^3 + 8 \cdot 10^2 + 7 \cdot 10 + 6 = 9 \cdot 1000 + 8 \cdot 100 + 7 \cdot 10 + 6$$

$$(3245)_8 = 3 \cdot 8^3 + 2 \cdot 8^2 + 4 \cdot 8 + 5 = \\ 3 \cdot 512 + 2 \cdot 64 + 4 \cdot 8 + 5 = 1536 + 128 + 32 + 5 = 1701$$

$$(101101)_2 = 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2 + 1 = \\ 32 + 8 + 4 + 1 = 45$$

Zapisi neoznačenih brojeva

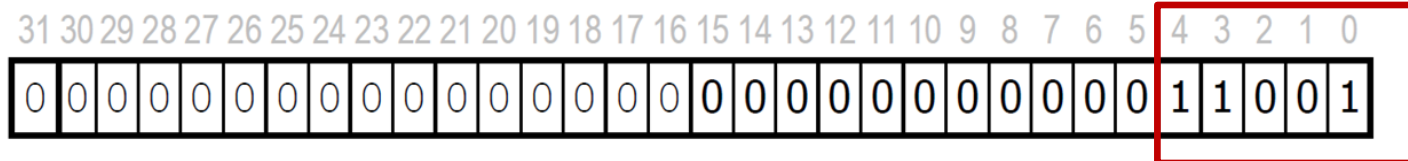
- ◆ Za predstavljanje neoznačenih brojeva sa n binarnih cifara u sistemu sa osnovom b ili $(\dots)_b^n$ važi ograničenje broja cifara i raspona brojeva koji se mogu zapisati

broj bitova	raspon
8	od 0 do 255
16	od 0 do 65535
32	od 0 do 4294967295

- ◆ Ukoliko je broj cifara potrebnih za zapis broja kraći od zadate dužine zapisa, onda se broj proširuje vodećim nulama, kao npr.
 $55 = (00110111)_2^8$

- ◆ Primer: Predstaviti broj 25 u 32-bitnom registru računara

- Korak 1: Konverzija broja u binarni sistem $(25)_{10} = (11001)_2$
- Korak 2: Prikazati sadržaj registra



Zapisi označenih brojeva

- ◆ Zapis označenih (*signed*) brojeva uključuje njegov znak (pozitivni ili negativni brojevi)
- ◆ Npr. sa 8 bitova može se zapisati 256 različitih brojeva ($2^8=256$), ali se moraju uzeti u obzir i negativne vrednosti brojeva
- ◆ Rešenje je da se predstave negativne vrednosti brojeva u opsegu $(-128 \div -1)$ i pozitivne vrednosti u opsegu $(0 \div 127)$
- ◆ U binarnom zapisu binarna cifra a_{n-1} na mestu najveće težine (MSB bit) označava znak broja i to je označena apsolutna vrednost:
 - broj je negativan ako je $a_{n-1} = 1$
 - broj je pozitivan ako je $a_{n-1} = 0$
- ◆ Primer: $+100 = (0\ 1100100)_2^8$ $-100 = (1\ 1100100)_2^8$

Zapisi označenih brojeva

◆ Načini predstavljanja označenih brojeva:

- 1) Znak i apsolutna vrednost
- 2) Nepotpuni komplement
- 3) Potpuni komplement

Znak i apsolutna vrednost:

Pozicija najveće težine ili krajnje levi bit (MSB) označava znak, a ostalih $n-1$ bitova apsolutnu vrednost broja:

- Ukoliko je krajnji levi bit **0** broj je pozitivan
- Ukoliko je krajnji levi bit **1** broj je negativan

+5	0	0	0	0	0	1	0	1
-5	1	0	0	0	0	1	0	1
	znak	apsolutna vrednost broja						

Znak i apsolutna vrednost

- ◆ Primer: Predstaviti broj -15 u zapisu znak i apsolutna vrednost

+15	0	0	0	0	1	1	1	1
-15	1	0	0	0	1	1	1	1

- ◆ Primer: Predstaviti brojeve 25 i -25 u zapisu znak i apsolutna vrednost
 $(25)_{10} = (11001)_2$

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
25:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	1
-25:	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	1

- ◆ Dva glavna nedostatka ovog načina zapisa su:
 - Pri izvođenju računskih operacija za otkrivanje eventualnog prekoračenja neophodno je ispitivati znak i apsolutnu vrednost oba argumenta
 - Nula se može zapisati na dva načina:

+0	0	0	0	0	0	0	0	0
-0	1	0	0	0	0	0	0	0

Nepotpuni komplement

- ◆ U zapisu broja u nepotpunom komplementu krajnji levi bit označava znak broja, dok ostalih $n-1$ bitova označavaju vrednost broja koja se zapisuje na sledeći način:
 - Za pozitivne brojeve kao apsolutna vrednost tog broja
 - Za negativne brojeve kao broj koji se dobija kada se u zapisu apsolutne vrednosti broja svaka cifra zameni njenim komplementom
- ◆ Ovaj postupak se naziva i komplement najveće cifre ili **komplement jedinice**
- ◆ Primer: Predstaviti broj -5 u obliku nepotpunog komplementa

+5 0 0 0 0 0 1 0 1

-5 1 1 1 1 1 0 1 0

Za zapis negativnog broja, svaka cifra je zamenjena njenim komplementom

Nepotpuni komplement

- ◆ Primer: Predstaviti brojeve 25 i -25 u 32-bitnom registru računara u formi nepotpunog komplementa ili komplementa jedinice

$$(25)_{10} = (11001)_2$$

25:

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	1

-25:

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	0	1	1	0

- ◆ Jedna od osnovnih mana ovog zapisa je dvoznačan zapis nule, što otežava izvođenje računskih operacija. Npr. u dekadnom sistemu na tri mesta nula se može zapisati kao 000 ili 999, a u binarnom sistemu na četiri mesta, nula se može zapisati kao 0000 ili 1111

+0	0	0	0	0	0	0	0	0
-0	1	1	1	1	1	1	1	1

- ◆ Kod promene znaka, potrebno je izvršiti komplementiranje svake cifre, odnosno svaku cifru oduzeti od najveće cifre sistema, bez obzira da li se radi o prelasku iz pozitivnog u negativan broj ili obratno

Potpuni komplement

- ◆ Zapis broja u potpunom komplementu omogućava jednostavnije izvođenje računskih operacija jer poseduje samo jedan zapis nule, što olakšava izvođenje operacije sa njom
- ◆ Ovaj postupak se naziva i komplement osnove ili **komplement dvojke**
- ◆ Pri zapisu broja u potpunom komplementu krajnje levi bit (MSB) označava znak broja, a ostalih $n-1$ označavaju vrednost broja:
 - Za pozitivne brojeve kao apsolutna vrednost broja
 - Za negativne kao broj koji se dobija kada se na zapis broja u nepotpunom komplementu doda jedinica na mesto najmanje težine (LSB)
- ◆ Primer: Predstaviti brojeve **-5** i **-15** u obliku potpunog komplementa

	+5	<table><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>1</td></tr></table>	0	0	0	0	0	1	0	1
0	0	0	0	0	1	0	1			
1) Nepotpuni komplement		<table><tr><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>0</td><td>1</td><td>0</td></tr></table>	1	1	1	1	1	0	1	0
1	1	1	1	1	0	1	0			
2)										
Potpuni komplement	-5	<table><tr><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>0</td><td>1</td><td>1</td></tr></table>	1	1	1	1	1	0	1	1
1	1	1	1	1	0	1	1			

	+15	<table><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td><td>1</td><td>1</td></tr></table>	0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1			
1) Nepotpuni komplement		<table><tr><td>1</td><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td></tr></table>	1	1	1	1	0	0	0	0
1	1	1	1	0	0	0	0			
2)		<table><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr></table>								
Potpuni komplement	-15	<table><tr><td>1</td><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td><td>0</td><td>1</td></tr></table>	1	1	1	1	0	0	0	1
1	1	1	1	0	0	0	1			

Binarna aritmetika

■ sabiranje

$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 0 = 1$$

$$1 + 1 = 0 \text{ sa prenosom } 1$$

$$\begin{array}{r} \overset{1}{1}00\overset{1}{1}0\overset{1}{1}1\overset{1}{1}1\overset{1}{1}.1101 \\ + \quad \quad 10101.11 \\ \hline 10101101.1001 \end{array}$$

$$\begin{array}{r} \overset{1}{1}\overset{1}{1}1\overset{1}{1}0\overset{1}{1}1\overset{1}{1}1\overset{1}{1}.1101 \\ + \quad \quad 10101.11 \\ \hline 10001101.1001 \end{array}$$

■ oduzimanje

pozicija	6	5	4	3	2	1
	1	0	0	1	1	1
-	0	0	1	0	1	1
	0	1	1	1	0	0

$$\begin{array}{r} 110 \\ - 011 \\ \hline 011 \end{array}$$

sa pozicije **6** se pozajmljuje jedinica i pozicija 5 dobija vrednost **10**, tj. 2, pozajmljuje se jedinica sa te pozicije i na poziciji 5 ostaje vrednost jedan, a na poziciji **4** je pozajmljena vrednost **10**

Zapis realnih brojeva

◆ **Realni brojevi** se u računarima predstavljaju kao:

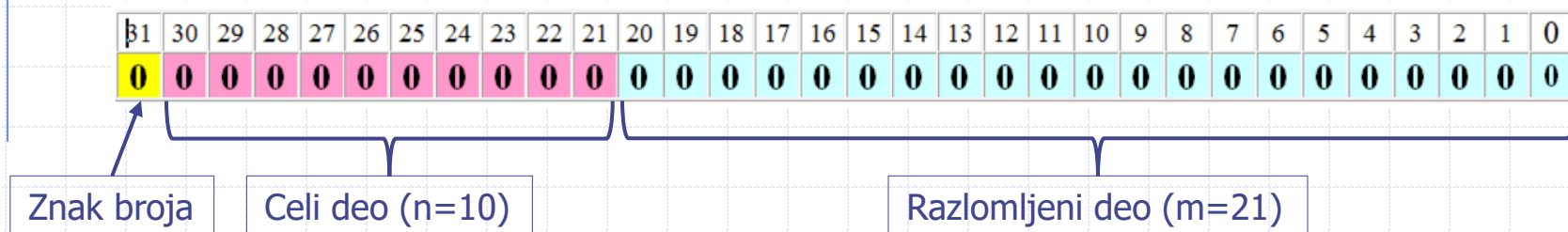
- Brojevi sa fiksnim zarezom (*fixed-point numbers*) kod kojih se unapred zna broj pozicija za razlomljeni deo
- Brojevi sa pokretnim zarezom (*float-point numbers*) kod kojih je moguće imati više ili manje pozicija za razlomljeni deo, i dele se na brojeve sa:
 - ◆ jednostrukom tačnošću (*float*)
 - ◆ dvostrukom tačnošću (*double*) gde je ostavljeno više bitova za razlomljeni deo

◆ U mnogim oblastima javljaju se i vrlo velike i vrlo male vrednosti brojeva sa kojima se vrše izračunavanja i često je potrebno koristiti format zapisa da pamti kako velike tako i male brojeve, pri čemu će relativna greška za sve brojeve biti jednaka

◆ Računarima se ne sme ostaviti nikakva proizvoljnost u pogledu toga kako će neki broj biti reprezentovan, pa se uvode **standardi** kojima se precizno definišu načini kako se pojedini formati brojeva zapisuju

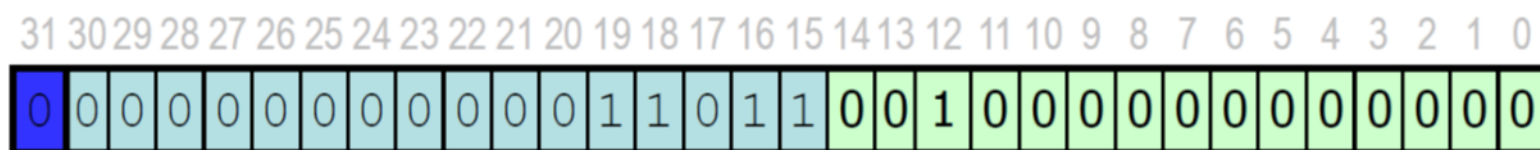
Brojevi sa fiksnim zarezom

- ◆ Zapis ovih brojeva se vrši na taj način što se m bitova koristi za predstavljanje celog dela broja, a n bitova za predstavljanje razlomljenog dela, dok se bit najveće težine (MSB) koristi za predstavljanje znaka broja



- ◆ Primer: Predstaviti broj 27,125 u 32-bitnom registru, pri čemu se za celobrojni deo koristi 16, a za razlomljeni deo 15 pozicija

$$(27.125)_{10} = (11011.001)_2$$



Brojevi sa pokretnim zarezom

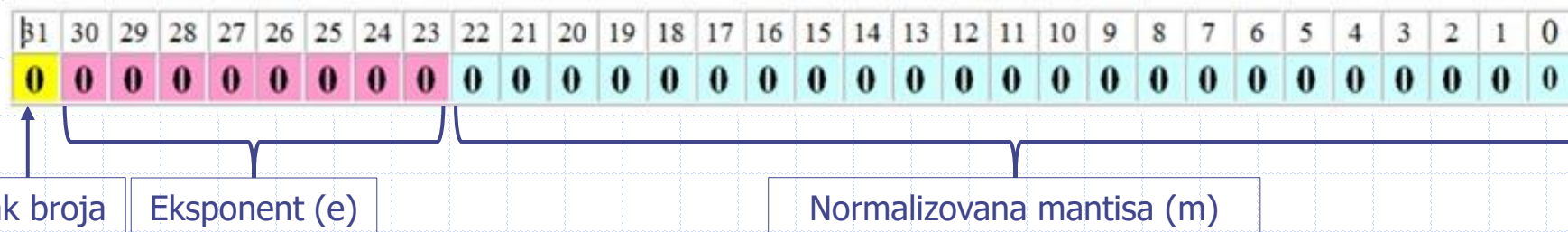
- ◆ Brojevima sa pokretnim zarezom se mogu predstaviti vrlo velike ili vrlo male vrednosti (npr. $1.0 \cdot 10^{25}$, $2.57 \cdot 10^{-33}$), pa se za ovaj format koristi eksponencijalni zapis

- ◆ Format eksponencijalnog zapisa je:

$$R = mb^e$$

R -vrednost broja, m -mantisa, b -osnova sistema (2), e -eksponent

- ◆ Svaki broj je moguće predstaviti na više načina u eksponencijalnom obliku, registrima dužine 32 ili 64 bita
- ◆ Normalizovana notacija podrazumeva položaj decimalnog zareza u mantisi neposredno iza prve nenulte cifre
- ◆ Eksponencijalni 32-bitni zapis broja (jednostruka tačnost):



Brojevi sa pokretnim zarezom

◆ Primeri zapisa:

dekadni broj $1200000000000 = 1.2 \times 10^{12}$

dekadni broj $0.0000000378 = 3.78 \times 10^{-8}$

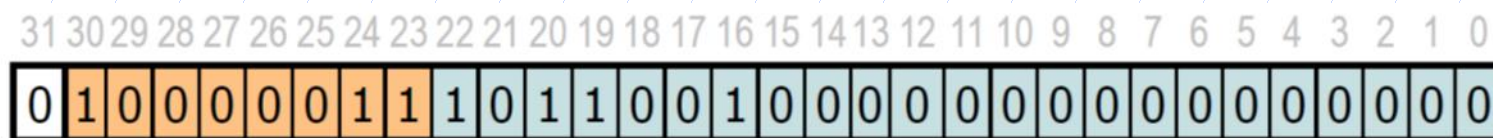
binarni broj $11010000000 = 1.101 \times 2^{10}$

◆ Prilikom zapisa negativnih celih vrednosti eksponenta u potpunom komplementu javlja se problem prilikom poređenja dve vrednosti, zato se eksponent ponderiše, tj. na stvarnu vrednost eksponenta se dodaje konstanta: 127 za float 1023 za double

◆ Primer: Predstaviti broj 27.125 u 32-bitnom registru u formatu pokretnog zareza

$$(27.125)_{10} = (11011.001)_2 = (1.1011001 \cdot 2^4)_2$$

$$\text{Eksponent se pamti sa pomeranjem: } e = (127 + 4)_{10} = (10000011)_2$$



Brojevi sa pokretnim zarezom

- Prilikom zapisa negativnih celih vrednosti eksponenta u potpunom komplementu javlja se problem prilikom poređenja dve vrednosti, zato se eksponent ponderiše, tj. na stvarnu vrednost eksponenta se dodaje konstanta: 127 za float 1023 za double

$$7.6875_{10} = 111.1011_2 = 1.111011 \times 2^2$$

$$S=0$$

$$E' = 2 + 127 = 129_{10} = 10000001_2$$

$$F = 11101100...00$$

$$-17.5_{10}$$

$$S=1$$

$$E' = 4 + 127 = 131_{10} = 10000011_2$$

$$F = 000110000...00$$

po standardu IEEE754 za **float**:

mantisa - 23 bita

eksponent - 8 bitova

jedan bit za znak broja

ukupna dužina - **32 bita**

za **double**:

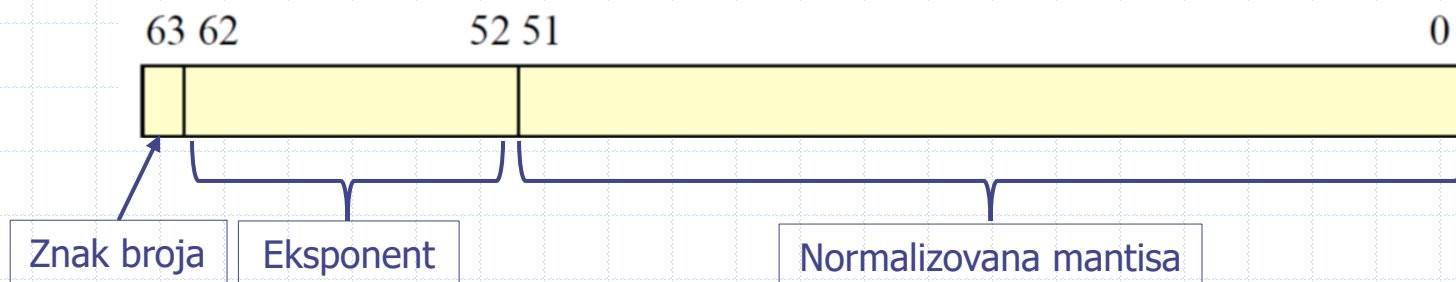
mantisa - 52 bita

eksponent - 11 bitova

jedan bit za znak broja

ukupna dužina - **64 bita**

- Za brojeve sa pokretnim zarezom dvostruke tačnosti koristi se 64-bitni zapis:



BCD kôd

- ◆ Pošto računari koriste binarnu aritmetiku, moguće je dekadne cifre kodirati binarnim vrednostima i onda ih smestiti u memoriju i nad ovako kodiranim brojevima direktno izvršavati aritmetičke operacije
- ◆ Ovaj način predstavljanja brojeva se naziva **BCD kôd** (*Binary Coded Decimals*) ili binarno kodirane dekadni sistem
- ◆ Za binarno kodiranje svake dekadne cifre potrebne su bar 4 binarne pozicije jer je $2^3 < 10 < 2^4$ ali se za neke kodove ovaj broj bitova povećava da bi kôd imao neke druge funkcionalnosti
- ◆ **Kôd 8421** se naziva prirodni BCD kôd jer se svaka dekadna cifra kodira svojom binarnom reprezentacijom i on je najčešće u upotrebi zato što se aritmetičke operacije nad ovako kodiranim brojevima najjednostavnije obavljaju

Dekadna cifra	8421	2421	XS3	2 od 5	9876543210	Hafmenov kod	Grejov kod	51111
0	0000	0000	0011	00011	0000000001	0000000	0000	00000
1	0001	0001	0100	00101	0000000010	1101001	0001	00001
2	0010	0010	0101	00110	0000000100	0101010	0011	00011
3	0011	0011	0110	01001	0000001000	1000011	0010	00111
4	0100	0100	0111	01010	0000010000	1001100	0110	01111
5	0101	1011	1000	01100	0000100000	0100101	0111	10000
6	0110	1100	1001	10001	0001000000	1100110	0101	11000
7	0111	1101	1010	10010	0010000000	0001111	0100	11100
8	1000	1110	1011	10100	0100000000	1110000	1100	11110
9	1001	1111	1100	11000	1000000000	0011001	1000	11111

Računarski podaci

3. Zapis teksta u računaru

Zapis teksta u računar

Tekst je "informacija namenjena ljudskom sporazumevanju koja može biti prikazana u dvodimenzionom obliku. Tekst se sastoji od grafičkih elemenata kao što su karakteri, geometrijski ili fotografski elementi ili njihove kombinacije, koji čine sadržaj dokumenta" (ISO standard definicija)

- ◆ Tekst se u računarima predstavlja kao jednodimenzionalni (linerni) niz karaktera ili znakova koji pripadaju unapred određenom fiksiranom skupu karaktera
- ◆ U zapisu teksta koriste se specijalni karakteri koji označavaju npr. tabulator, prelazak u novi red, razmak između reči, kraj teksta itd.
- ◆ Pošto se računari baziraju na binarnoj aritmetici, osnovna ideja je da se svakom karakteru može pridružiti ceo broj na unapred dogovoren način, i te brojeve zovemo **kodovima karaktera**

Zapis teksta u računarima, EBCDIC, ASCII

- ◆ Tokom razvoja računarstva broj karaktera je postajao sve veći
- ◆ Na početku je englesko govorno područje bilo dominantno, pa su se koristili sledeći karakteri:
 - Velika slova engleskog alfabeta : A,B,...,Z
 - Mala slova engleskog alfabeta : a,b,...,z
 - Cifre : 0,1,...,9
 - Interpunkcijski znaci : . , : ; ? ! + * - _ / itd.
 - Specijalni znaci : kraj reda, tabulator itd.
- ◆ Za ove karaktere su se 70-ih godina pojavile kôdne tabele ili strane standardnih karakterskih kôdova i to:
 - **EBCDIC** (*Extended Binary Coded Decimal Interchange Code*) je IBM-ov standard, korišćen na mainframe računarima i bušenim karticama, mogao je predstaviti 256 karaktera sa po 8 binarnih cifara (osmobitni kôd)
 - **ASCII** (*American National Standard Code for Information Interchange*) kôd je predstavljen 1968. god. od strane Američkog nacionalnog instituta za standarde, kao sedmobitni kôd, u kojem se može predstaviti 128 karaktera

ASCII kôd

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174	|	
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177		DEL

ASCII kôd

Osobine 7-bitnog ASCII standarda:

- Ukupan broj karaktera koji se mogu predstaviti ASCII kôdom je $2^7=128$ (od 0-127)
- Karakteri u ASCII kôdu se čuvaju i prenose u grupi od 8 bitova, pri čemu se bit najveće težine koristi za kontrolu parnosti
- Prva 32 karaktera, od $(00)_{16}$ do $(1F)_{16}$ su specijalni kontrolni karakteri npr. ETX-kraj teksta, HT-tabulator, STX-Početak teksta, BEL-zvučno upozorenje, EOT-kraj prenosa ili kraj fajla...
- Ukupno 95 karaktera ima pridružene grafičke likove (*printable characters*)
- Cifre 0-9 predstavljene su kôdovima od $(30)_{16}$ do $(39)_{16}$, tako da se njihov ASCII zapis jednostavno dobija dodavanjem prefiksa 011 na njihov binarni zapis
- Kôdovi velikih i malih slova se razlikuju u samo jednom bitu u binarnoj formi, pa se npr. veliko slovo A kodira brojem $(41)_{16}$ odnosno $(1000001)_2$, dok se malo slovo a kodira brojem $(61)_{16}$ odnosno $(1100001)_2$
- Slova su poređana u kolacionu sekvencu u skladu sa engleskim alfabetom

ASCII kôd

- Primer:

String	H	e	l	l	o
ASCII Value	72	101	108	108	111
Binary	01001000	01100101	01101100	01101100	01101111

- U praksi postoje 8-bitna proširenja ASCII standarda od strane ISO ali i od Microsofta, pri čemu se pojedinačni ASCII znakovi zapisuju korišćenjem celih bajtova (8 bitova), i takav zapis ima mesta za dodatnih 128 znakova ($2^8=256$)
- Termin „kodna stranica“ (*code page*) odnosi se na konkretnu proširenu ASCII tabelu koja se koristi u određenom računarskom sistemu
- U okviru kodnih strana, pozicije iznad 127 koriste se za dodavanje podrške za specijalne znakove opšteg tipa ili za definisanje znakova različitih svetskih pisama
- Za azijske jezike nije dovoljno 256 mesta za zapis svih karaktera

ASCII kôd

Kodna stranica **1251** (koja je ranije bila korišćena na Windows-u), odgovara znakovima srpskog ćiriličnog pisma, dok se za prikaz latiničnih znakova iz slovenskih jezika koristi kodna stranica **1250**, dok se za grčki alfabet koristi kodna stranica **737**

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
2	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{		}	~	□
8	Ѓ	Г	Д	Ђ	Ѕ	Ї	Љ	Њ	Ћ	Ќ	Ќ	Ќ	Ќ	Ќ	Ќ	Ќ
9	Ѓ	Г	Д	Ђ	Ѕ	Ї	Љ	Њ	Ћ	Ќ	Ќ	Ќ	Ќ	Ќ	Ќ	Ќ
A	У	Ў	Ј	Ѣ	Ѓ	Ѓ	Ѓ	Ѓ	Ѓ	Ѓ	Ѓ	Ѓ	Ѓ	Ѓ	Ѓ	Ѓ
B	Ѓ	Ѓ	Ѓ	Ѓ	Ѓ	Ѓ	Ѓ	Ѓ	Ѓ	Ѓ	Ѓ	Ѓ	Ѓ	Ѓ	Ѓ	Ѓ
C	А	Б	В	Г	Д	Е	Ж	З	И	Й	К	Л	М	Н	О	П
D	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ъ	Ы	Ь	Э	Ю	Я
E	а	б	в	г	д	е	ж	з	и	й	к	л	м	н	о	п
F	р	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я

230 - ж
158 - ћ
247 - ч
248 - ш

232 - ч
230 - ć
154 - š
158 - ž



među pozicijama istovetnih znakova u kodnim rasporedima 1251 i 1250, nema poklapanja

Unicode standard

- ◆ Danas se uglavnom koriste drugi standardi za prikaz teksta, koji su razvijeni iz razloga što je broj karaktera je postajao sve veći
- ◆ Standardi **ISO 10646**, **Unicode** i **UTF**
- ◆ **Unicode** je sveprisutni i opšteprihvaćeni standard za kodiranje znakova u računarskim sistemima
- ◆ Unicode standard je formiran sa nekoliko osnovnih ideja:
 - glavni cilj je obuhvatiti sve znakove, iz svih svetskih pisama tj. koliko god je moguće znakova
 - svaki znak mora imati jedinstvenu kodnu poziciju (*code point*) što je pristup koji je idejno sličan ASCII standardu
 - Unicode specifikacija mora biti potpuno kompatibilna sa ASCII specifikacijom (kodovi znakova iz ASCII tabele odgovaraju kodnim pozicijama iz Unicode specifikacije)
- ◆ Standard Unicode se može shvatiti kao "ASCII modernog doba"
- ◆ Unicode standardom se može prikazati ~1.1 milion karaktera, a najnoviji Unicode 15.1 (2023 g.) koristi ~150.000 kodnih pozicija

Unicode standard

- ◆ U okviru Unicode standarda znakovi se kodiraju prema dogovoru, tako da svakom znaku odgovara jedinstvena kodna pozicija čime je otklonjena zbrka koju je (ranije) unosilo korišćenje različitih kodnih stranica:
 - prvih 128 pozicija zauzimaju redom znakovi iz ASCII tabele
 - zatim slede pisma evropskih jezika, sa znakovima koji zauzimaju kodne pozicije bliže početnim ASCII pozicijama (latinica, ćirilica, grčki i ruski alfabet itd.)
 - više pozicije zauzimaju manje uobičajena pisma sa velikim brojem znakova (kinesko, japansko, korejsko...)
 - najviše pozicije zauzimaju razni specijalni znaci (znaci muzičke notacije, emotikoni..)
- ◆ ASCII znakovi zauzimaju 7 bitova, dok sa Unicode znakovima zahteva, shodno kodnim pozicijama, više od jednog bajta za zapis (dva, tri, ili četiri)

č	-	268
Σ	-	8721
✓	-	9989
😬	-	128578

Primer nekoliko znakova čije kodne pozicije prevazilaze vrednost 127

UCS-2 i UTF standardi

- ◆ ISO definiše **UCS-2** standard koji svaki Unicode karakter prevodi u odgovarajuća dva bajta (2B)
- ◆ Tekstovi kodirani preko UCS-2 standarda sadrže veliki broj nula, koje u UNIX operativnom sistemu i u programskom jeziku C imaju specijalno značenje
- ◆ **UTF** (*Unicode Transformation Format*) je algoritam koji svakom Unicode karakteru dodeljuje određeni niz bajtova čija dužina varira od 1 do najviše 6
- ◆ Najčešće korišćena varijanta ovog algoritma je **UTF-8** koja je dovoljna za zapis svih dvobajtnih Unicode karaktera, i njegova dužina varira od 1 do najviše 3 bajta
- ◆ ISO je uveo u upotrebu i **UTF-16**, **UTF-32**, kao i standard **UCS-4**
- ◆ Format UTF-8 se danas najčešće koristi za zapis tekstualnih datoteka i kodiranje web dokumenata

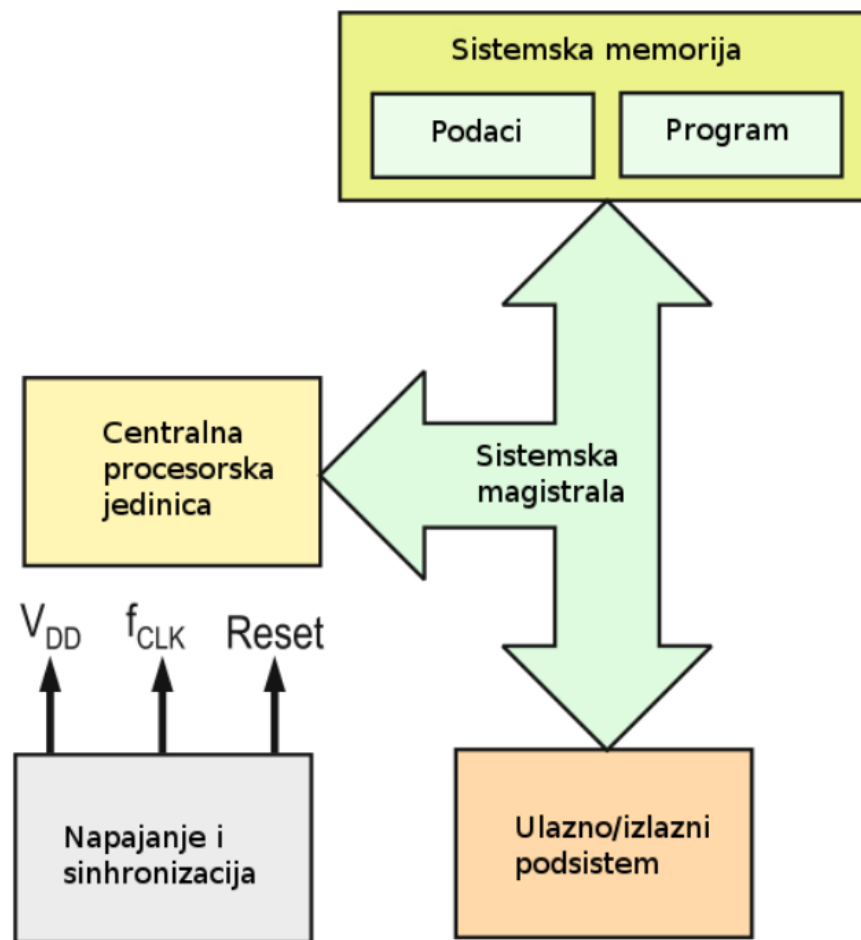
Primer:

	H	e	l	l	o
Unicode	U+0048	U+0065	U+006C	U+006C	U+006F
UTF-8	48	65	6C	6C	6F
UTF-16	00 48	00 65	00 6C	00 6C	00 6F
ASCII	48	65	6C	6C	6F

Računarski podaci

4. Obrada podataka u računar

Arhitektura računara



Blok-šema računarskog sistema zasnovanog na
Fon Nojmanovoj arhitekturi

Arhitektura računara

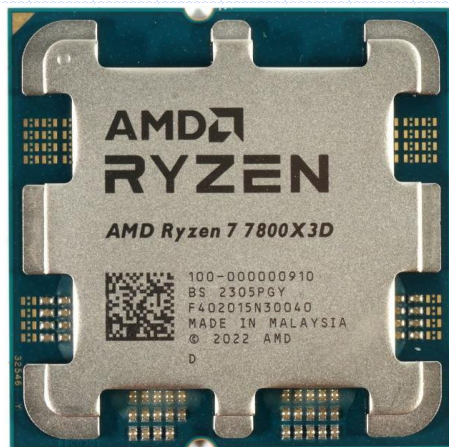
- ◆ Računarski sistem zasnovan na **Fon Nojmanovoj arhitekturi** (programi i podaci su smešteni u isti memorijski prostor) sastoji se od sledećih komponenti:
 - **Centralna procesorska jedinica** (CPU, procesor, mikroprocesor): Predstavlja ključni deo svakog računarskog sistema, čita instrukcije iz programske memorije, dekoduje ih i izvršava. CPU je zadužen za adekvatno korišćenje perifernih jedinica koje sačinjavaju ulazno/izlazni podsistem (I/O) računara
 - **Sistemska memorija** (RAM memorija) zadužena je za čuvanje programa i podataka koji se koriste od strane procesora. Računar sadrži dva tipa memorije: programsku memoriju i memoriju podataka
 - ◆ U programsku memoriju smešta se program u formi sekvence mašinskih instrukcija koju procesor izvršava. Program određuje funkcionalnost celokupnog sistema.
 - ◆ Memorija za podatke ima ulogu čuvanja podataka (promenljivih, varijabli) koji se obrađuju tokom izvršavanja programa.

Arhitektura računara

- **Ulazno/Izlazni podsystem (I/O)** računara uključuje sve komponente, tj. periferni uređaje koji omogućavaju procesoru da razmenjuje informacije sa drugim komponentama sistema i vrši interakciju sa spoljašnjim okruženjem
- **Sistemske magistrale** predstavljaju skup linija koje povezuju procesor, memoriju i ulazno/izlazni podsystem. Postoje grupe linija koje obavljaju različite funkcije unutar sistema: adresna magistrala, magistrala podataka i kontrolna magistrala
- ◆ U slučaju sistema baziranih na procesorima opšte namene, osnovne komponente sistema (CPU, memorija, ulazno/izlazni uređaji) su odvojena integrisana kola ili čipovi.
- ◆ Ova elektronska kola se montiraju na višeslojnu PCB štampanu ploču (matična ploča) u okviru kojih su povezana provodnim vezama
- ◆ Računari su danas u različitim formama: PC (desktop), laptop, tablet, mainframe, ugrađeni (embedded), hibridni, prenosivi, industrijski...

Procesor računara

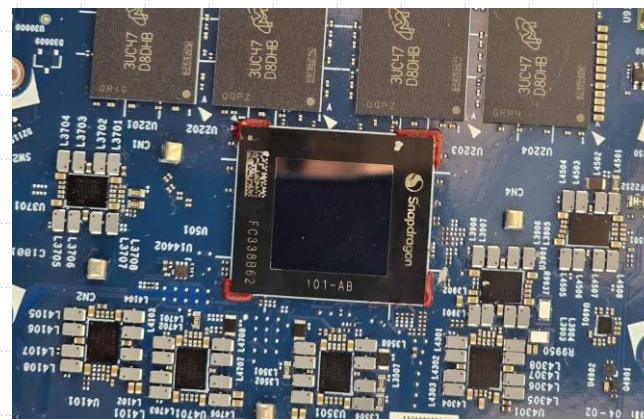
AMD Ryzen 7 7800X3D
procesor za PC



Intel Core i7 12700KF procesor za PC



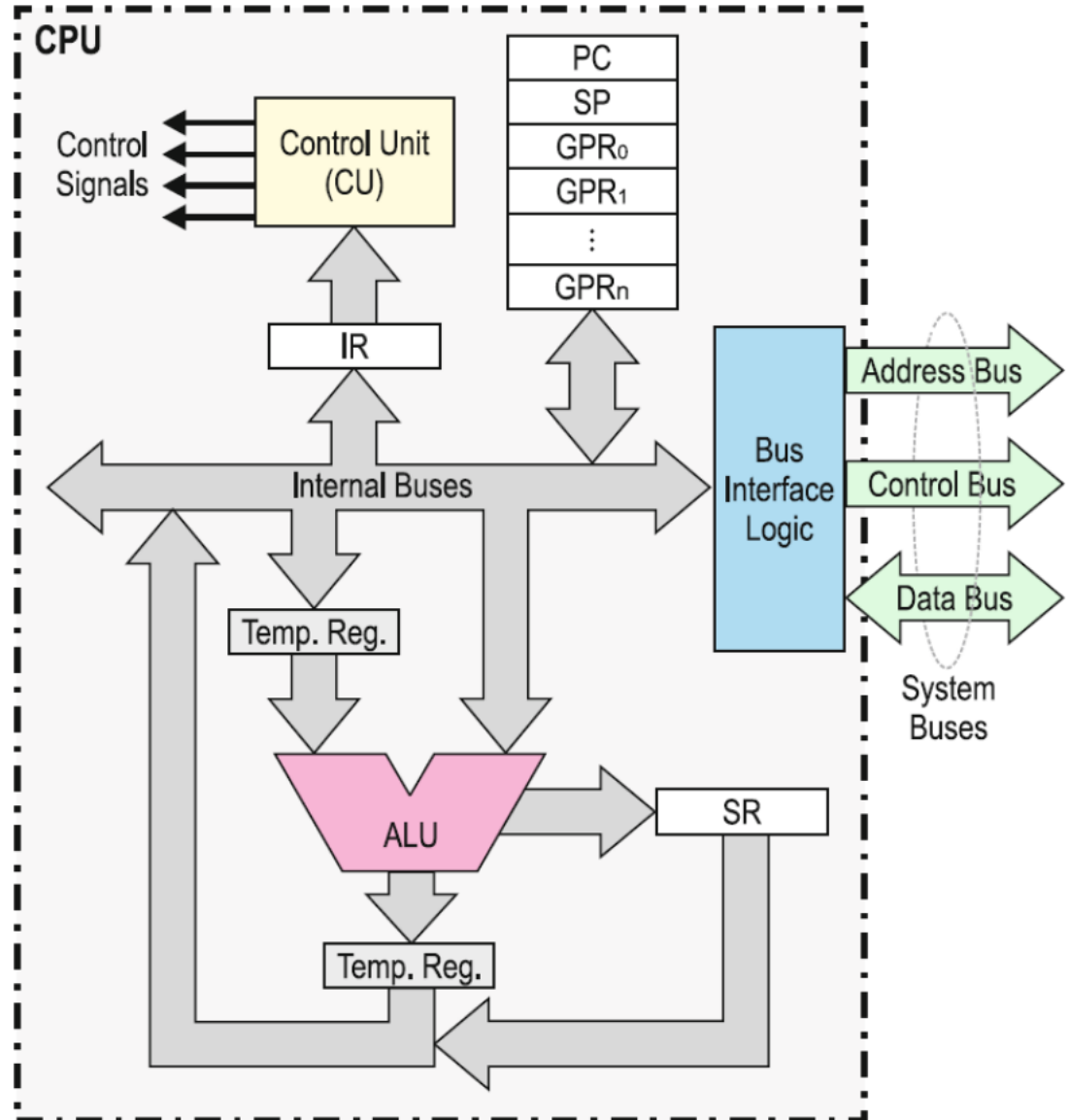
AMD Epic 7543 procesor
za server



Snapdragon procesor za mobilne
uređaje na matičnoj ploči

Arhitektura procesora

Blok-šema arhitekture procesora (CPU)



Arhitektura procesora

- ◆ Centralna procesorska jedinica **CPU** (*Central Processing Unit*) predstavlja najvažniji deo računara
- ◆ CPU je zadužen za izvršavanje instrukcija, kao i za generisanje odgovarajuće sekvence signala koja proizvodi akciju koja se izvršava na hardveru računarskog sistema
- ◆ Hardverske komponente procesora čine:
 - Aritmetičko-logička jedinica **ALU** (*Arithmetic Logic Unit*)
 - Kontrolna jedinica **CU** (*Control Unit*)
 - Skup registara
 - Logika za implementaciju sprežnog interfejsa **BIL** (*Bus Interface Logic*)
- ◆ Softverske komponente procesora čine:
 - Skup mašinskih instrukcija
 - Modovi (načini) adresiranja
- ◆ Arhitektura procesora zasniva se na dužini registara tako da danas postoje dva tipa:
 - 32-bitni procesori (danas se retko se koriste!)
 - 64-bitni procesori

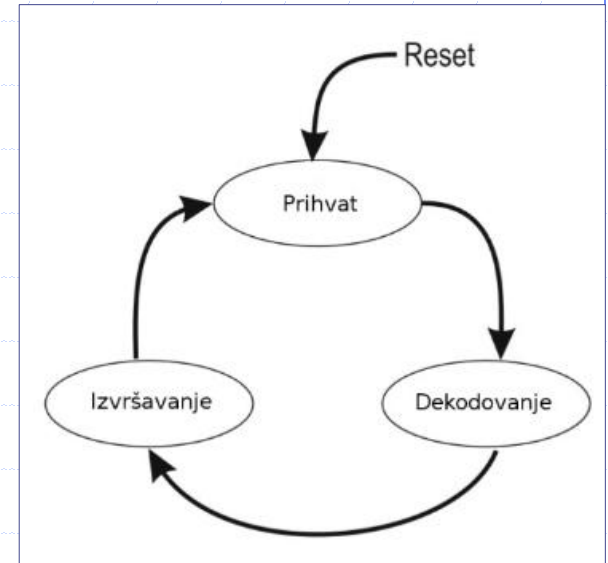
Arhitektura procesora - CU

Kontrolna jedinica (CU) upravlja funkcionalnošću procesora implementirajući konačni automat FSM (*Finite State Machine*) koji ciklično ponavlja tri stanja:

1) **Prihvat instrukcije**: Tokom stanja prihvata nove instrukcije, instrukcija se iz programske memorije učitava u CPU. Programski brojač (PC) obezbeđuje adresu u memoriji sa koje se instrukcija čita. Adresirana instrukcija se preko magistrale podataka prebacuje u registar instrukcija (IR)

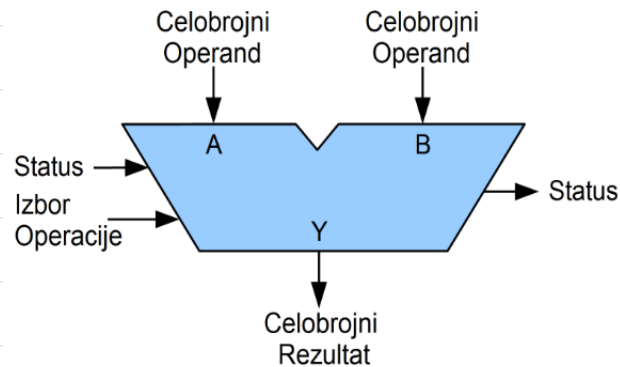
2) **Dekodiranje instrukcije**: Nakon prihvata instrukcije, CU prelazi u stanje dekodiranja, u kome se značenje instrukcije „dešifruje“, a dekodirana informacija se koristi kako bi se slali odgovarajući upravljački signali ka CPU komponentama u cilju izvršavanja aktivnosti predviđenih samom instrukcijom

3) **Izvršavanje instrukcije**: U ovom stanju, CU šalje komande odgovarajućim funkcionalnim jedinicama u okviru CPU u cilju izvršavanja aktivnosti određenih instrukcijom. Na kraju ove faze, sadržaj programskog brojača se inkrementira, kako bi pokazivao na adresu sledeće instrukcije u programskoj memoriji, koja će se izvršiti tokom narednog instrukcijskog ciklusa



CPU ciklus: Fetch-Decode-Execute (FDE)

Arhitektura procesora - ALU



Aritmetičke operacije:

- Sabiranje (sa ili bez prenosa)
- Oduzimanje (sa ili bez pozajmice)
- Inkrement
- Dekrement
- Drugi komplement
- Množenje
- Deljenje

Logičke operacije:

- Bitsko I (AND)
- Bitsko ILI (OR)
- Bitsko EKS-ILI (XOR)
- Bitska negacija (NOT)

Operacije pomeranja:

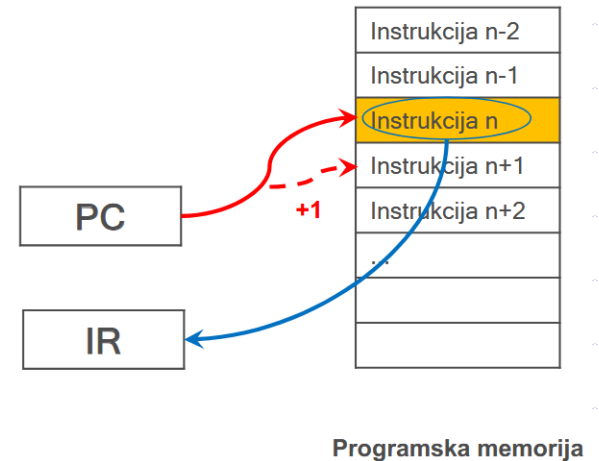
- Pomeranje (ulevo ili udesno)
- Rotacija (ulevo ili udesno)

Arhitektura procesora - registri

- ◆ Registri procesora omogućavaju privremeno smeštanje podataka, memorijskih adresa i kontrolnih informacija, tako da im se može brzo pristupiti. Oni predstavljaju najbržu memoriju u računaru, ali najmanjeg kapaciteta
- ◆ CPU registri se mogu podeliti u 2 grupe:
 - 1) **Registri opšte namene GPR**: Registri koji nisu "vezani" za specifične funkcije procesora i koriste se za smeštanje podataka, promenljivih ili pokazivača na adrese. Nazivaju se još i adresni ili registri za podatke. U zavisnosti od arhitekture procesora, CPU može imati do nekoliko desetina GPR registara
 - 2) **Registri specijalne namene SFR**: Registri koji su zaduženi za specifične funkcije u okviru CPU, a navažniji registri su:
 - Registar instrukcija (**IR**)
 - Programski brojač **PC** (*Program Counter*) ili pokazivač instrukcija **IP** (*Instruction Pointer*)
 - Pokazivač steka **SP** (*Stack Pointer*)
 - Statusni registar (**SREG**), koji sadrži indikatore „zastavice“ (**Flags**)

Arhitektura procesora - registri

- Registar instrukcija (**IR**) prima instrukcije iz programske memorije, kako bi CU jedinica mogla da ih dekodira i izvrši. Akcija prenosa instrukcije iz memorije u IR naziva se prihvatanje instrukcije.
- Programski brojač (**PC**) sadrži adresu instrukcije koja se učitava iz programske memorije u instrukcijski registar. Svaki put kada se instrukcija prihvata i dekoduje, automatski se inkrementira vrednost PC registra, kako bi pokazivao na narednu instrukciju koja će se izvršavati tokom sledećeg ciklusa. Izuzetak predstavljaju instrukcije skoka i poziva potprograma.
- Struktura **steka** realizuje se u okviru memorije za podatke, a pokazivač steka (SP) sadrži adresu na kojoj se nalazi podatak koji se trenutno nalazi na vrhu steka (stek se popunjava od nižih ka višim adresama ili obrnuto, a prilikom svakog dodavanja novog podatka na stek, ili preuzimanja podatka sa steka, SP se automatski inkrementira ili dekrementira po **LIFO** principu)
- **Statusni registar** sadrži skup indikatorskih bita (Flags), koji se automatski setuju/resetuju u zavisnosti od rezultata aritmetičke ili logičke instrukcije koju obavlja ALU



Memorija računara

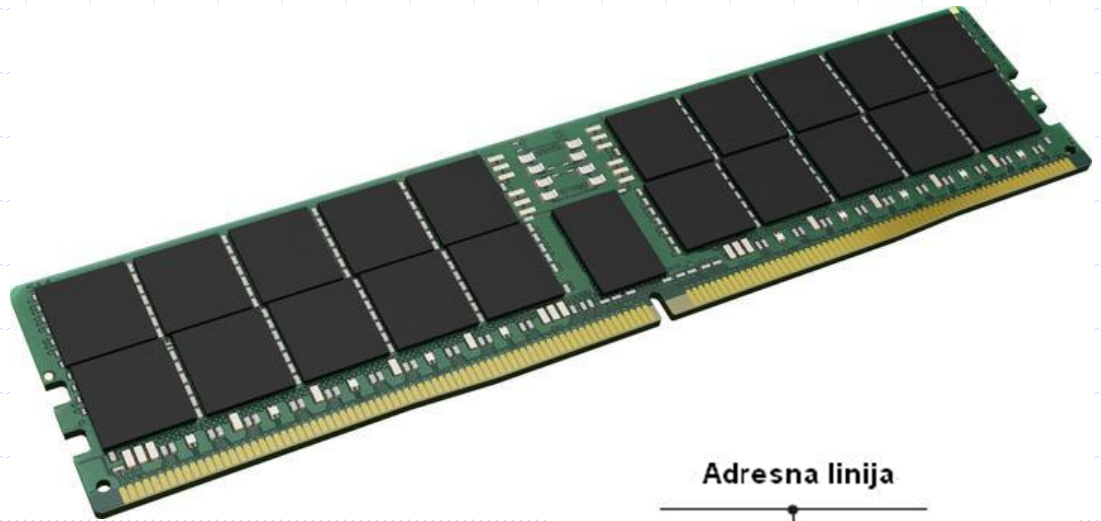
- ◆ Memorija računara ili sistemska memorija je mesto na kom se nalaze programi koji se izvršavaju i podaci koji se obrađuju
- ◆ Postoji više vrsta memorija koje koristi računarski sistem ali se pod pojmom „memorija” najčešće podrazumeva sistemska, glavna ili operativna memorija
- ◆ Osnovna podela memorija:
 - Privremena, nepostojana (*volatile*) - čuva podatke samo dok ima napajanja (RAM memorije)
 - Stalna, postojana (*non-volatile*) - podaci ostaju u memoriji i po isključivanju računara (statičke memorije, hard-diskovi, fleš-memorije...)
- ◆ Memorija se sastoji od velikog broja bistabilnih elemenata od kojih svaki skladišti po jedan bit informacije, i ovi bitovi su organizovani u n-bitne grupe koje funkcionišu kao registri, a nazivaju se još i memorijskim lokacijama
- ◆ Definisanje lokacije na memorijskom medijumu može se ostvariti na osnovu:
 - adrese pridružene lokacije (adresni pristup)
 - na osnovu sadržaja lokacije (asocijativni pristup)

Memorija računara

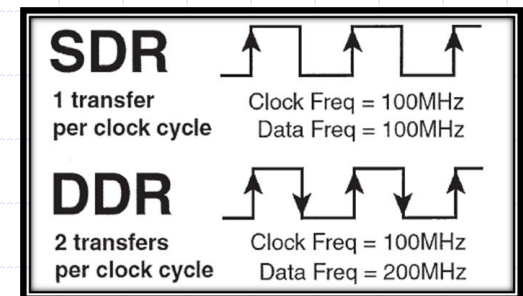
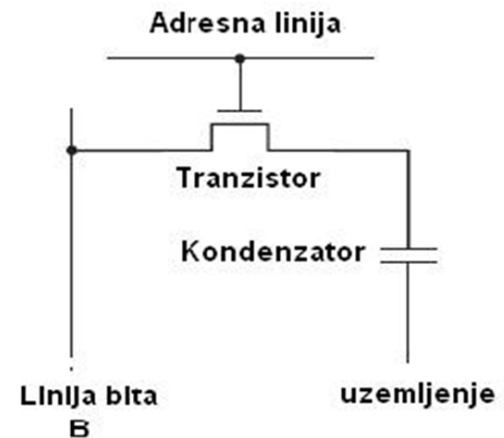
- ◆ Osnovna podela poluprovodničkih memorija je:
 - **RAM** (*Random Access Memory*), memorije sa slučajnim pristupom (upis/čitanje)
 - **ROM** (*Read Only Memory*), memorije samo za čitanje
- ◆ Memorijske ćelije su u formi bipolarnih i MOS tranzistora, pri čemu one bipolarnog tipa imaju kraće vreme pristupa, ali su MOS memorije manjih dimenzija i jeftinije. Za oba tipa memorijskih ćelija važi da se njihov sadržaj gubi nakon isključivanja napajanja računara
- ◆ Prema načinu čuvanja podataka, RAM memorije se dele na:
 - Statičke poluprovodničke memorije (**SRAM**)
 - Dinamičke poluprovodničke memorije (**DRAM**)
- ◆ Kod RAM memorija pojedinačna ćelija sadrži 1 tranzistor i 1 kondenzator, i neophodno je periodično osvežavanje sadržaja (zbog toga se nazivaju dinamičke)
- ◆ Savremene računarske memorije su tipa **DDR** (*Double Data Rate*) što znači da su to **SDRAM** sa dvostrukom brzinom prenosa podataka
- ◆ Dvostruko veće performanse postižu se prenosom podataka 2 puta u jednom ciklusu: jednom na prednjoj (rastućoj) ivici, a drugi put na zadnjoj (opadajućoj) ivici takt signala

DDR memorija računara

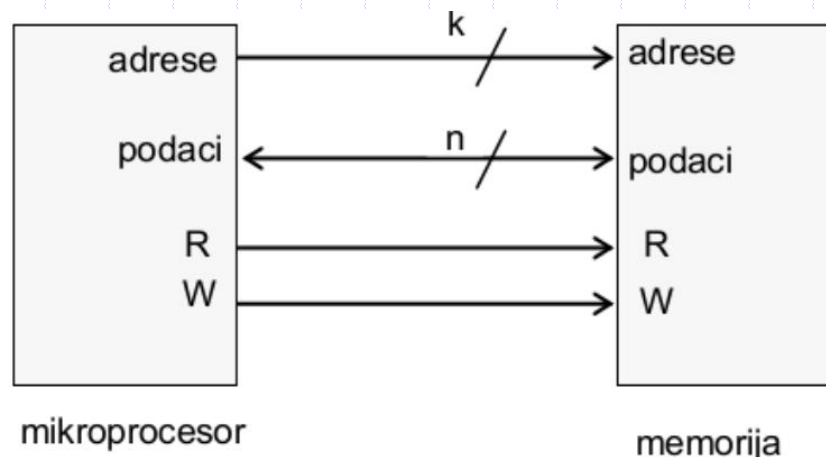
DDR5 memorijski modul



- ◆ Trenutna tehnologija računarskih memorija je **DDR5**
- ◆ DRAM memorijsku ćeliju čini kombinacija jednog tranzistora i jednog kondenzatora
- ◆ Kondenzator ima malu kapacitivnost i vrlo brzo gubi naelektrisanje, pa je potrebno periodično obnavljanje upisanih podataka (osvežavanje), i zato naziv „dinamičke“
- ◆ Posebna elektronika za osvežavanje zahteva složenije memorijske čipove, dok mali broj tranzistora u ćeliji čini DRAM module jeftinim za proizvodnju

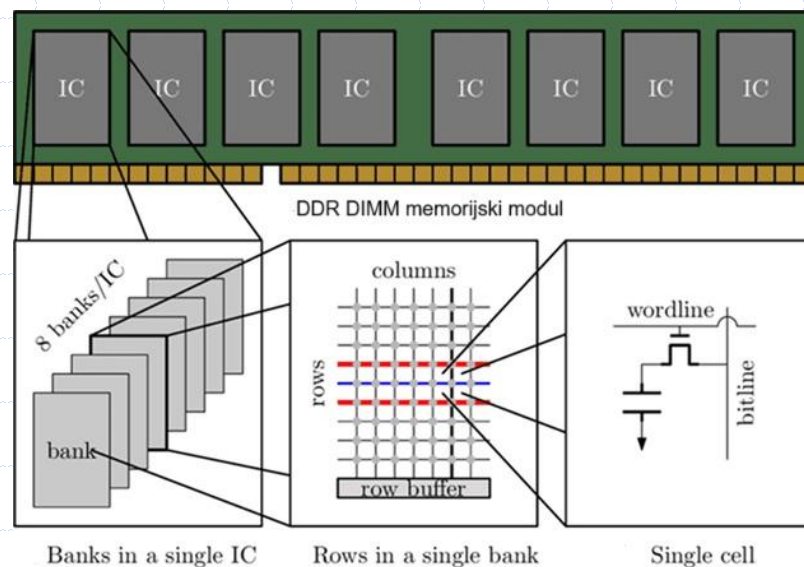


DDR memorije



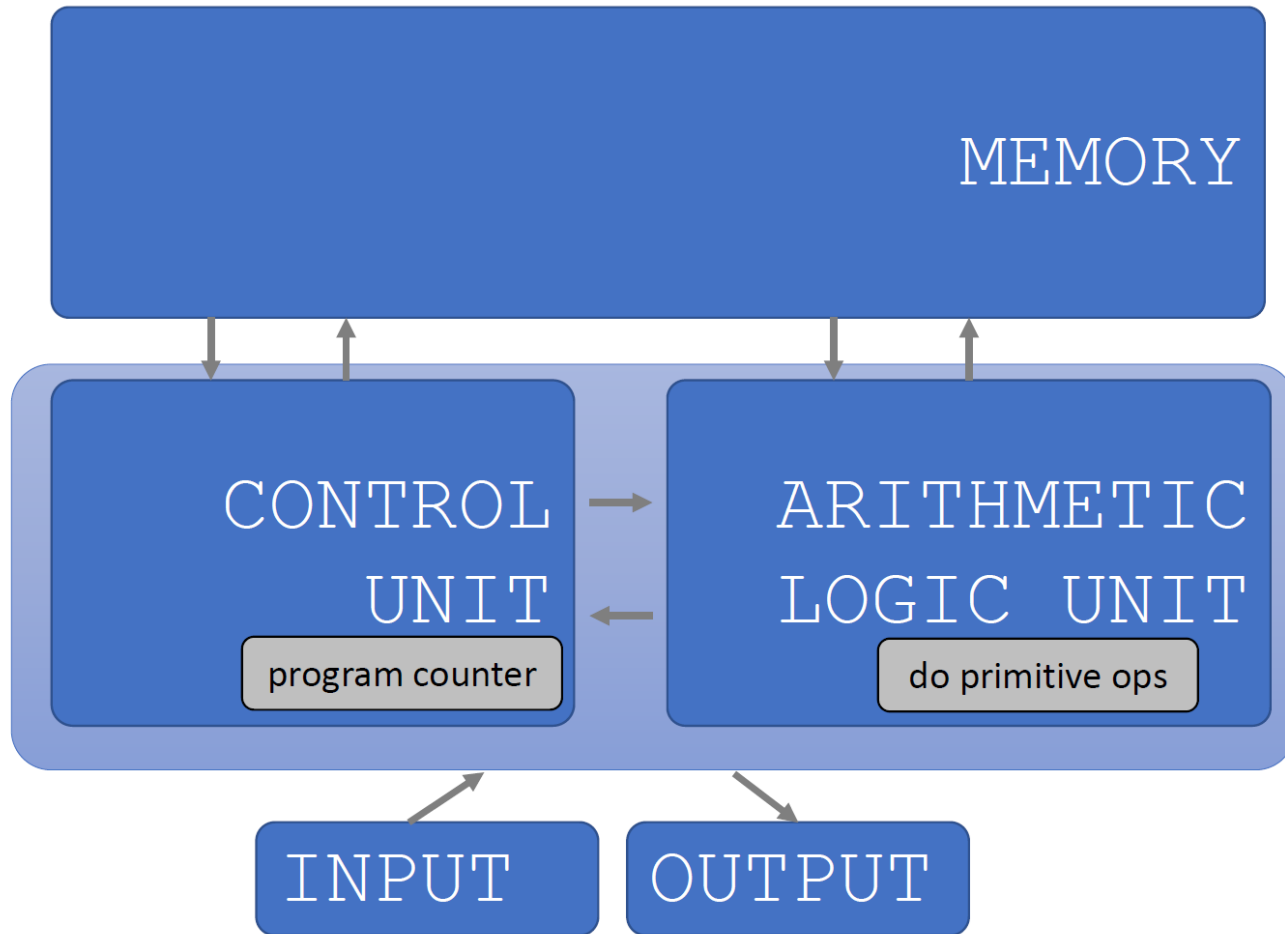
- ◆ Za povezivanje memorije sa procesorom koriste se grupe linija koje se nazivaju magistralama (bus):
- **Adresna magistrala** (*Address Bus*): linije preko kojih procesor zadaje adresu lokacije kojoj se pristupa
 - **Magistrala podataka** (*Data Bus*): linije preko kojih se vrši prenos podataka od procesora ka memoriji kod operacije upisa, odnosno od memorije ka procesoru prilikom čitanja
 - **Kontrolna magistrala** prenosi signale za dozvolu čitanja R (*Read*) i dozvolu upisa W (*Write*), neki memorijski moduli poseduju dodatni kontrolni ulaz CS (Chip Select), preko kojeg se dozvoljava odnosno zabranjuje pristup modulu

DDR memorije



- ◆ Trenutni razvoj DDR memorija fokusira se na povećanje njihovog propusnog opsega, smanjenje kašnjenja, potrošnje energije, manje zagrevanje i jednostavniju i jeftiniju izradu
- ◆ Ciklus upisa počinje tako što procesor simultano postavlja ispravnu adresu na linije adresne magistrale i podatak na magistralu podataka a nakon toga aktivira upravljački signal za upis. Kada memorijski kontroler prepozna da je signal upisa postao aktivan, pokreće operaciju upisa
- ◆ Signal za ciklus čitanja aktivira se nakon što procesor postavi ispravnu adresu na linije adresne magistrale. Kada memorijski kontroler prepozna da je signal čitanja postao aktivan, pokreće operaciju čitanja

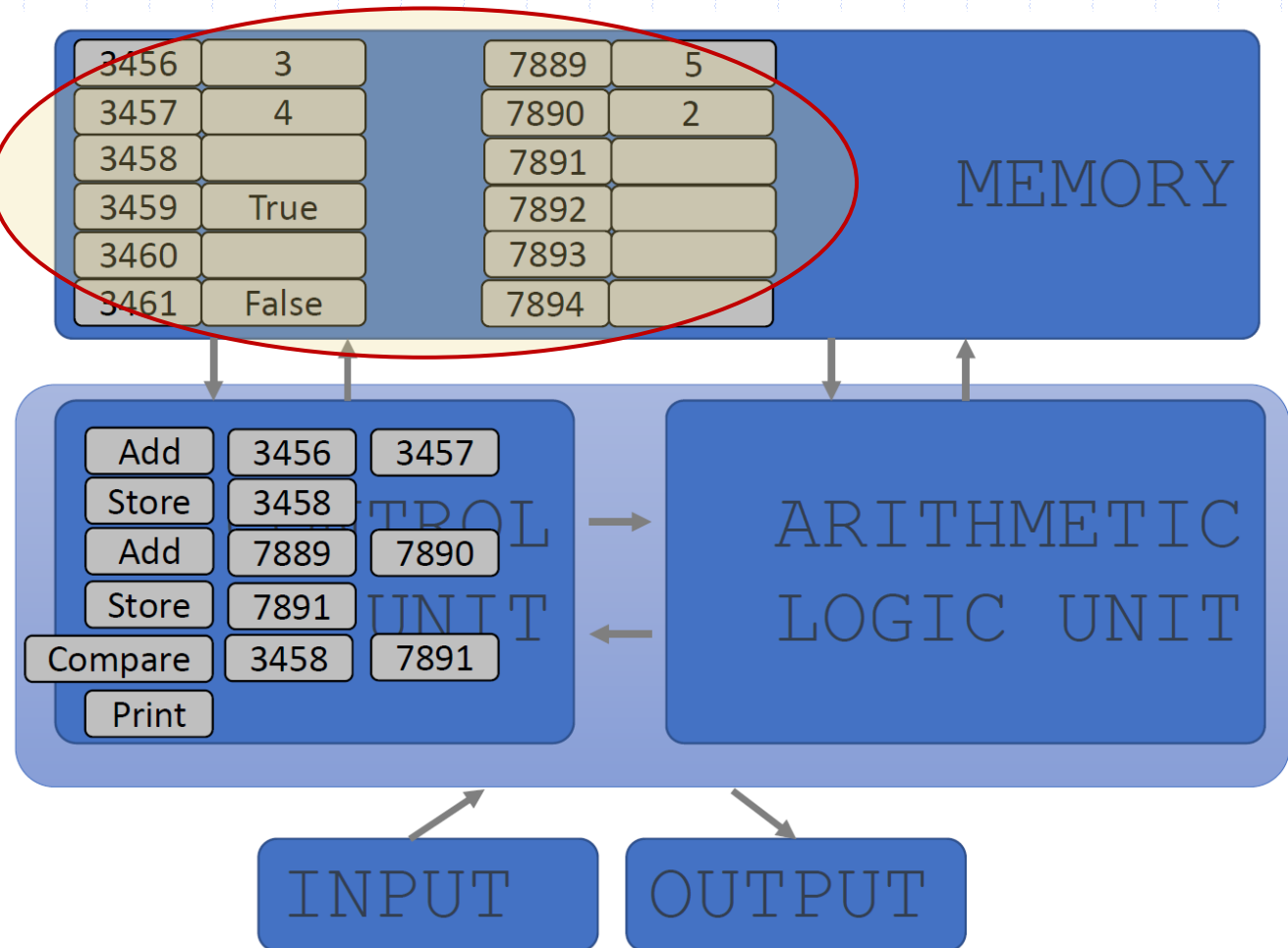
Izvršavanje programa - primer



Arhitektura računara

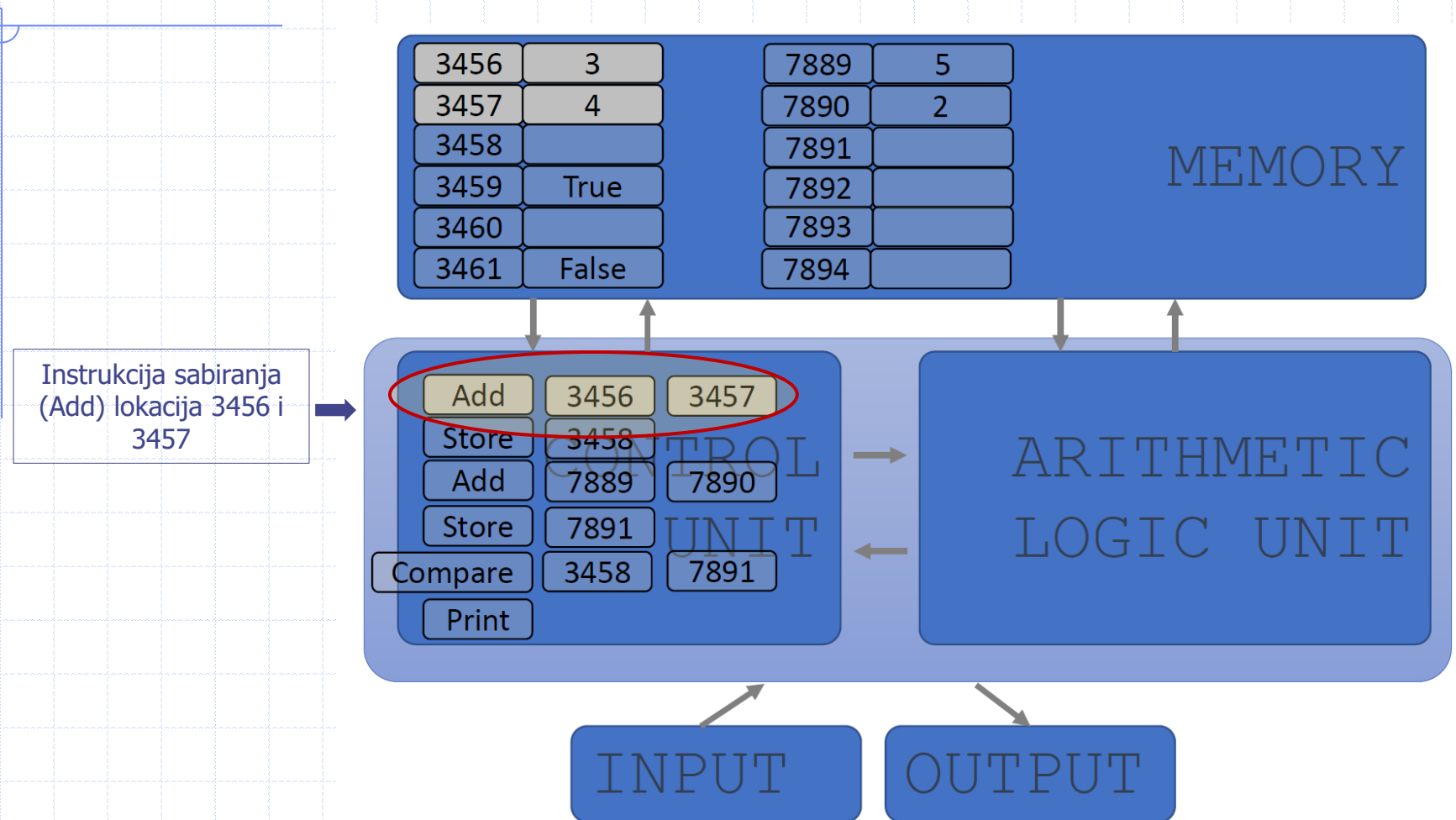
Izvršavanje programa - primer

Memorijske lokacije
sa adresama
(3456...3461 i
7889...7894) i
vrednostima (2, 3, 4,
5, True, False)



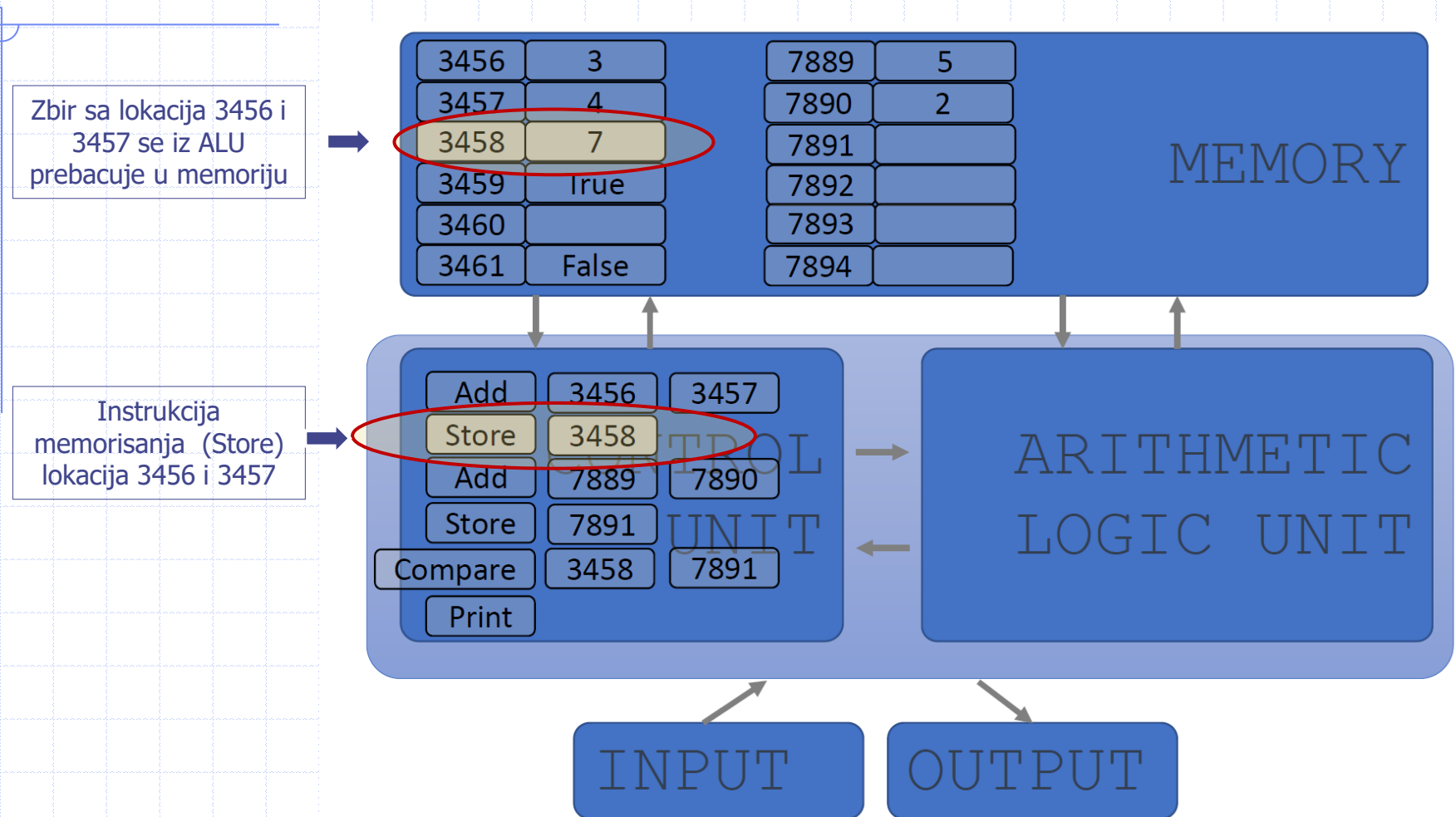
Na osnovu progrmskih instrukcija vrši se obrada podataka, izvršavanje tih instrukcija prema njihovom redosledu vrši poseban program INTERPRETER

Izvršavanje programa - primer



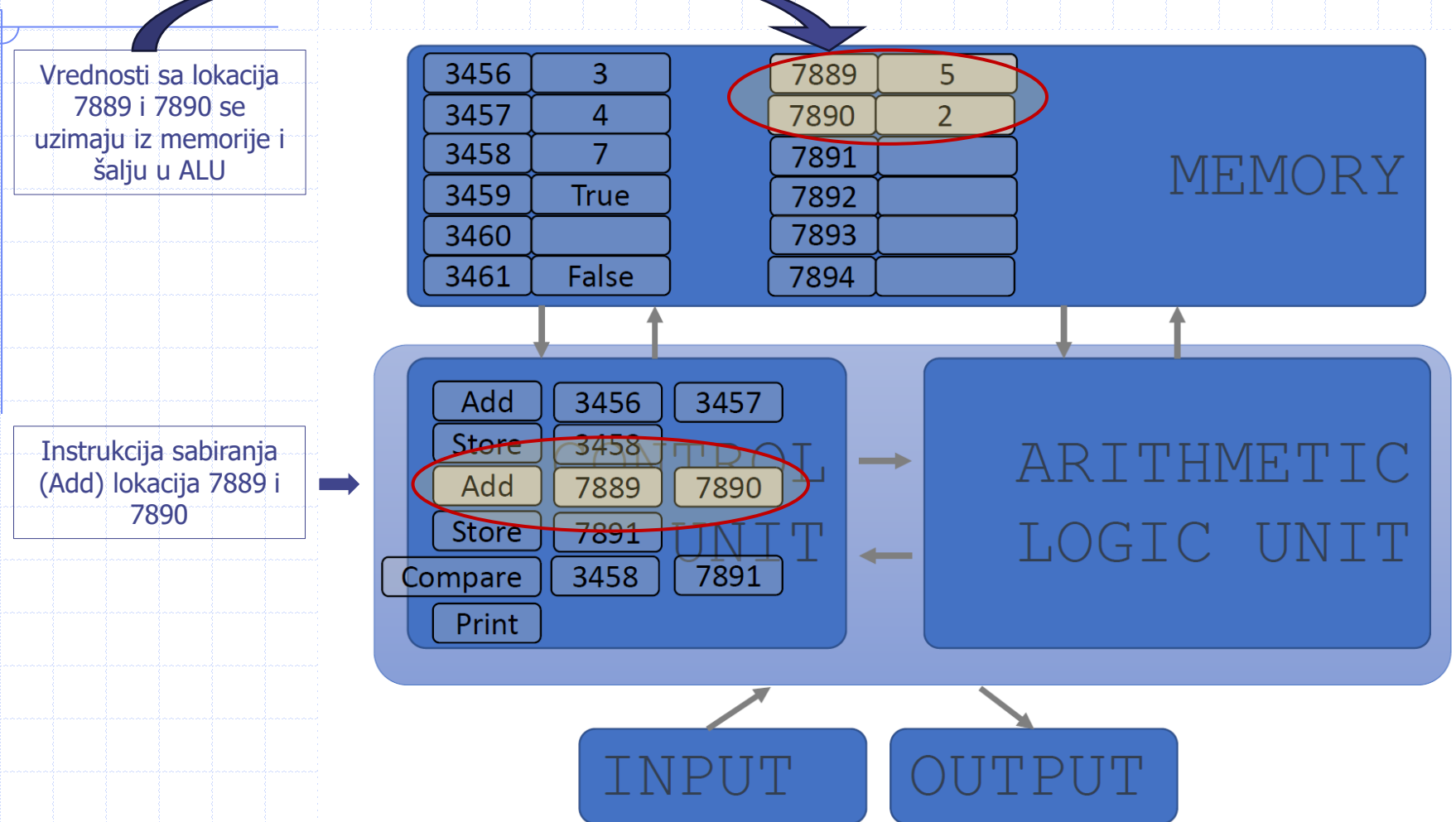
Program sa instrukcijama se nalazi u kontrolnoj jedinici, i kada se pokrene prvi put interpreter „vidi“ prvu instrukciju (Add-sabiranje) 2 vrednosti na memorijskim lokacijama 3456 i 3457, uzimaju se vrednosti sa te dve lokacije i šalju u ALU gde se vrši sabiranje

Izvršavanje programa - primer



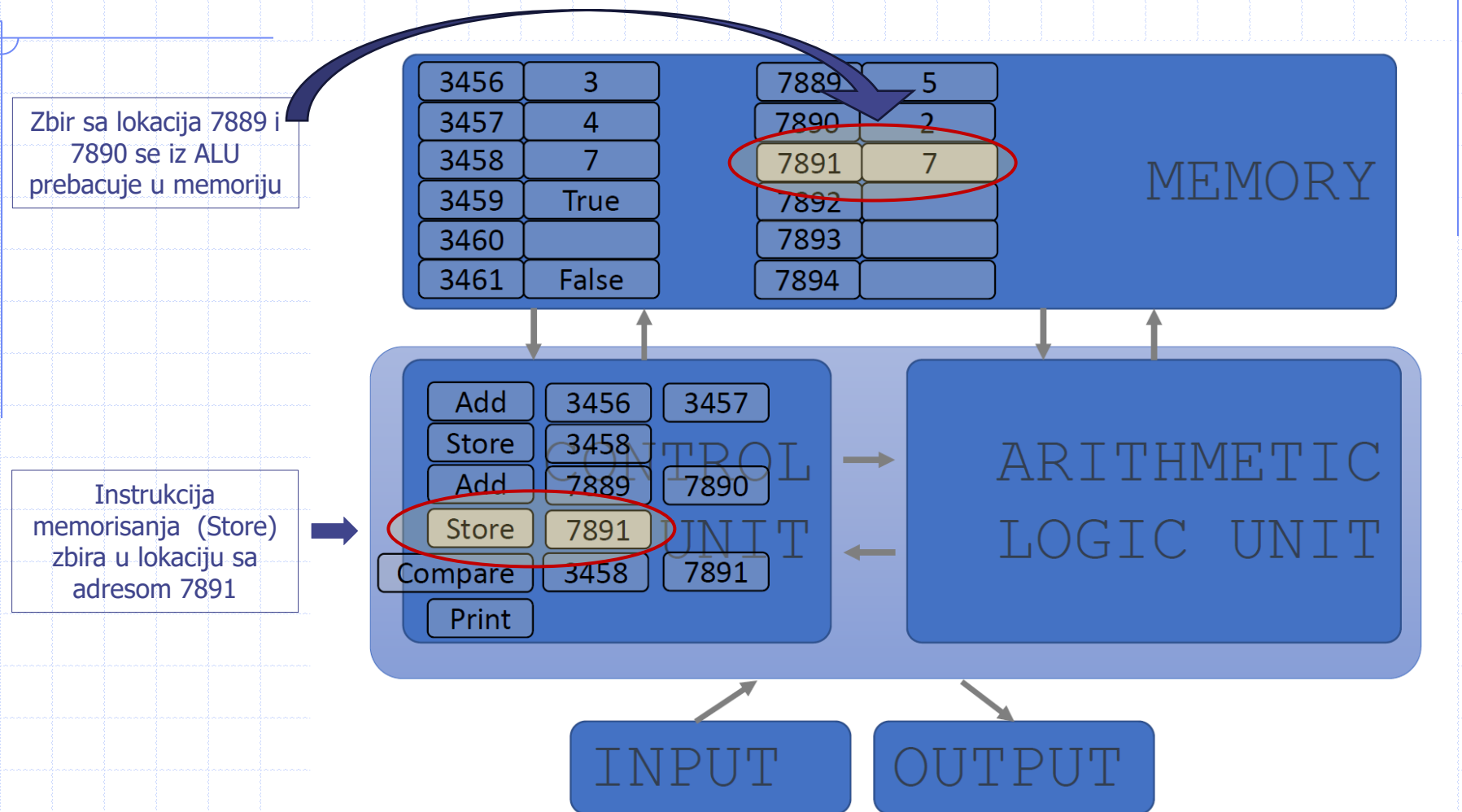
Sledeći korak je negde sačuvati rezultat zbira ($3+4=7$) pa se komandom STORE ovaj ovaj zbir uzima iz ALU i memoriše na lokaciji 3458

Izvršavanje programa - primer



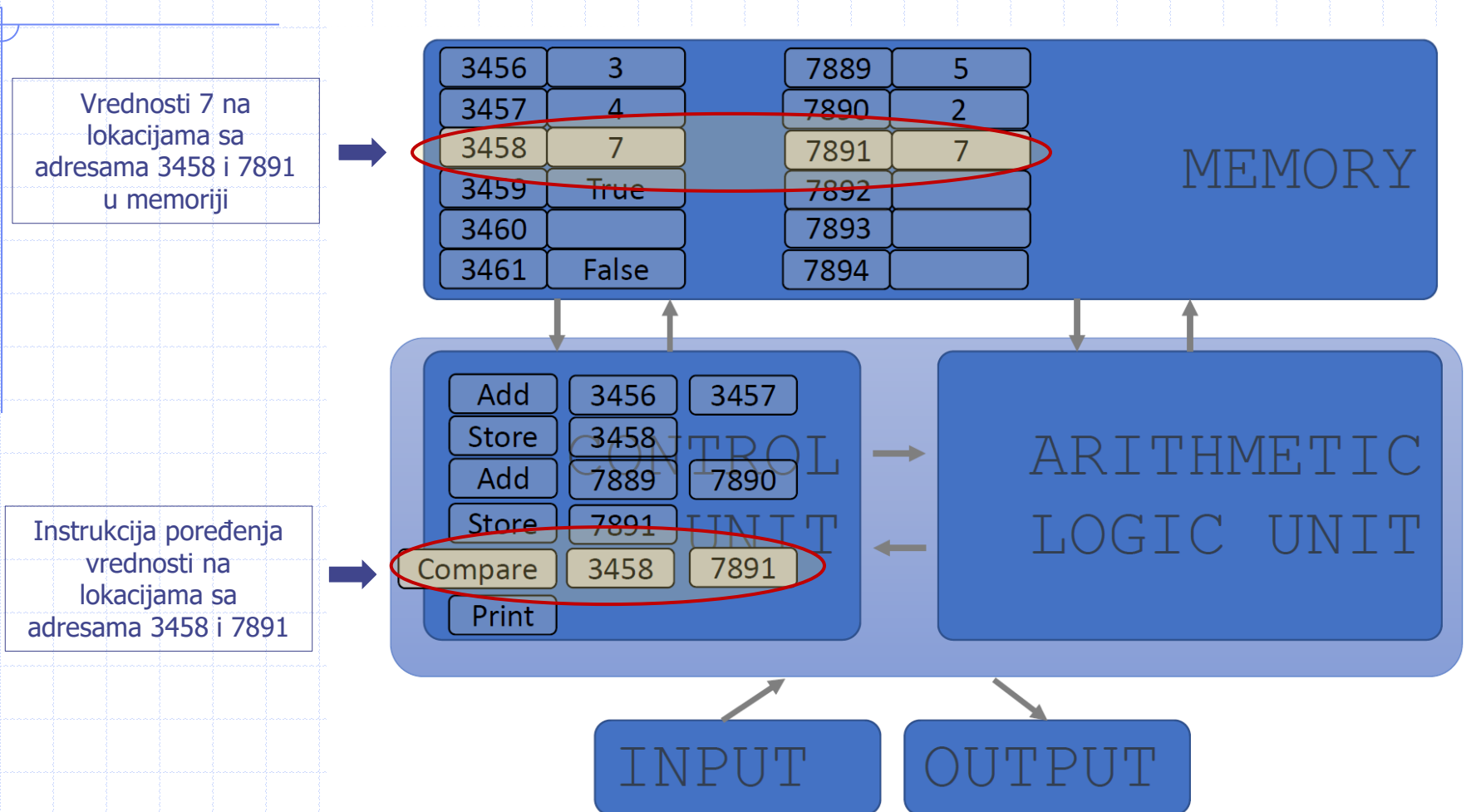
Sledeća komanda je sabiranja vrednosti sa memorijskih adresa 7889 i 7890, vrednosti se uzimaju iz memorije i šalju u ALU gde se vrši njihovo sabiranje

Izvršavanje programa - primer



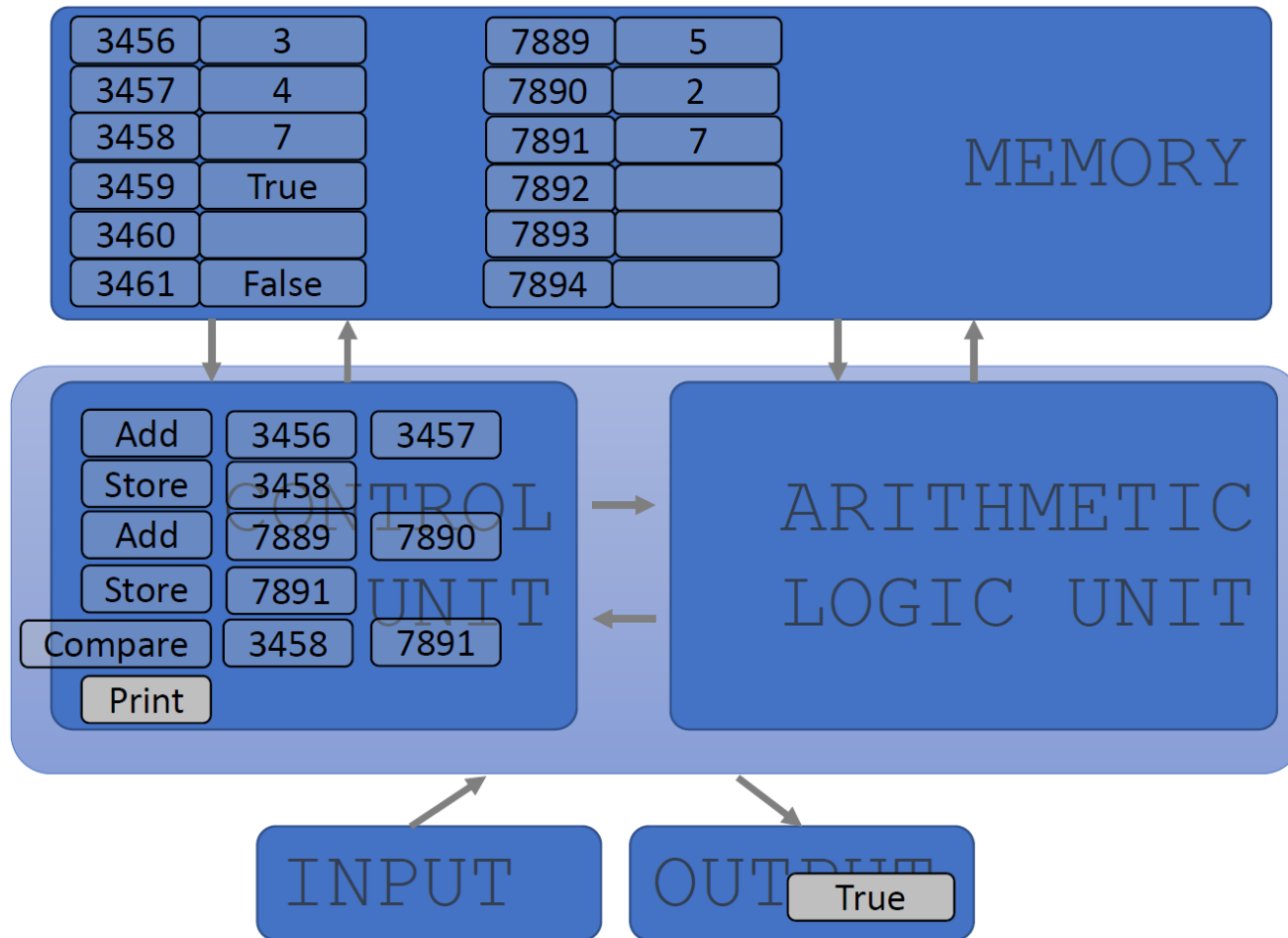
Sledeća komanda je sačuvati rezultat zbira ($5+2=7$) pa se komandom STORE ovaj ovaj zbir uzima iz ALU i memoriše na lokaciju 7891

Izvršavanje programa - primer



Sledeća komanda je poređenje rezultata dva zbira zbira, pa se komandom COMPARE porede vrednosti sa lokacija 3458 i 7891, ALU vrši njihovo poređenje i vraća vrednost True interpreteru

Izvršavanje programa - primer



Poslednja komanda je komanda Print za prikaz rezultata poređenja vrednosti na lokacijama 3458 i 7891, kao izlaz se prikazuje True jer su vrednosti jednake

